컴퓨팅 사고력을 키우는

이산수학

3판

박주미 지음

한빛아카데미 Hanbit Academy, Inc.

CHAPTER 08

그래프

목차

8.1 그래프의 개념

8.2 그래프의 종류

8.3 그래프의 표현

8.4 오일러와 해밀턴

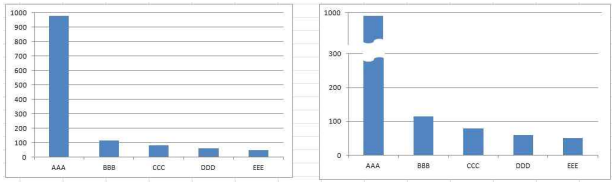
8.5 그래프의 활용

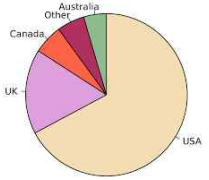
미리보기

그래프란?

❖ **그래프(도표)는 직선, 곡선, 도형 등 그래픽의 요소에 의해 시각화된 차트를 말하며 넓은 뜻으로는 다이어그램도 포함한다.**

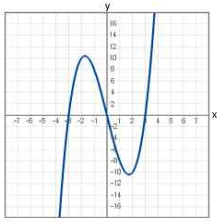
- 통계자료를 특정한 형식으로 표현한 그림 : 막대, 선, 띠, 원 그래프

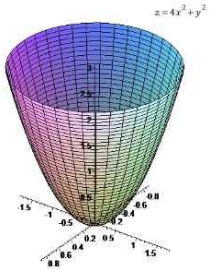




- 함수의 그래프: 주어진 함수가 나타내는 직선이나 곡선을 말한다.

$f : X \to Y$인 함수 f의 그래프 $G = \{(x, f(x)) : x \in X\}$





❖ **이산수학과 컴퓨터공학 분야에서 다루는 추상적 개념의 그래프 (자료 구조)**

❖ **실생활에서 서로 관련되는 두 가지 또는 그 이상의 양의 관계를 정점과 간선을 이용하여 그림으로 나타낸 것이다.**

❖ **꼭지점의 집합과 변의 집합을 이루는 순서쌍이다.**

그래프의 응용 분야

❖ **한붓그리기로 잘 알려진 쾨니히스베르크의 다리 건너기 문제 이후에 그래프 이론은 컴퓨터, 화학, 산업공학, 전자공학, 언어학, 유전학, 사회과학, 경제학 등 많은 분야에 이용되고 있다.**

- 통신 네트워크 전송 문제
- 소셜 네트워크(social network)의 분석 : SNS 친구 관계
- 논리 회로의 설계 및 분석
- 최단 경로 찾기

❖ **작업 계획의 분석, 분자 구조식 설계와 화학 결합의 표시, 유한 오토마타, 지하철 노선도, 자동차 네비게이션 시스템, 지도에서 정점들을 색칠하는 문제, 회로도를 평면 회로도로 구현할 수 있는지 등에 이용된다.**

학습목표

- 그래프의 기본 개념과 표기 방법을 이해한다.
- 다양한 종류의 그래프를 이해한다.
- 정점, 변, 차수, 영역 상이의 관계를 이해하고 활용한다.
- 오일러 그래프와 해밀턴 그래프를 구분하고 경로와 회로를 찾는다.
- 인접행렬과 인접리스트를 이용해 그래프를 표현한다.
- 최단 경로를 구하는 다익스트라 알고리즘을 이해한다.
- 깊이 우선 탐색과 너비 우선 탐색을 구별하여 탐색한다.

8.1 그래프의 개념

8.1.1 그래프의 정의와 표현

정의 8-1 **그래프(graph : $G=(V,E)$)**

공집합이 아닌 **정점**(vertex, node)의 집합 V와 서로 다른 정점의 쌍 (v_i, v_j)를 연결하는 **변**(edge)의 집합 E로 구성된 구조

$$G=(V,E)$$
$$V=\{v_1, v_2, \cdots, v_n\}$$
$$E=\{e_1, e_2, \cdots, e_m\}=\{(v_i, v_j), \cdots\},\ 1 \le i, j \le n$$

- 정점 v_i와 v_j는 서로 **인접**(adjacent)한다 : 정점 v_i와 v_j를 연결하는 변이 존재한다.
- 변 e_k는 정점 v_i와 v_j에 **근접**(incident)한다 : 변 e_k는 정점 v_i와 v_j를 연결한다.

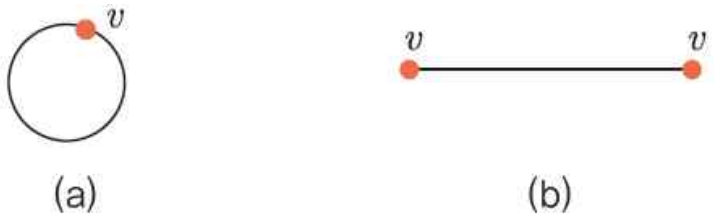
- $G=(V,E)$: 정점의 집합 V와 변의 집합 E로 구성되는 그래프 G
- $V=\{v_1, v_2, \ldots, v_n\}$: 그래프 G를 구성하는 정점의 집합 V
- $E=\{e_1, e_2, \ldots, e_m\}$: 그래프 G를 구성하는 변의 집합 E

  $=\{(v_i, v_j), \ldots\}$: 변의 집합 E의 각 원소가 근접하는 정점의 쌍

8.1.1 그래프의 정의와 표현

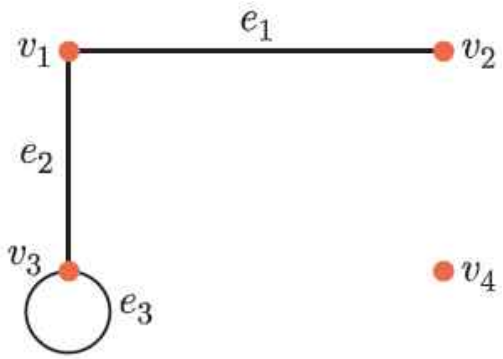


[그림 8-1] 방향성이 있는 변의 순서쌍 표현



[그림 8-2] 루프의 표현

8.1.1 그래프의 정의와 표현



- 그래프 G는 정점의 집합 V와 변의 집합 E로 구성
- 집합 V는 v_1, v_2, v_3, v_4로 구성
- 집합 E는 e_1, e_2, e_3로 구성
  - $e_1 = (v_1, v_2)$ 또는 (v_2, v_1)
  - $e_2 = (v_1, v_3)$ 또는 (v_3, v_1)
  - $e_3 = (v_3, v_3)$

- $G = (V, E)$
- $V = \{v_1, v_2, v_3, v_4\}$
- $E = \{e_1, e_2, e_3\} = \{(v_1, v_2), (v_1, v_3), (v_3, v_3)\} = \{(v_2, v_1), (v_3, v_1), (v_3, v_3)\}$

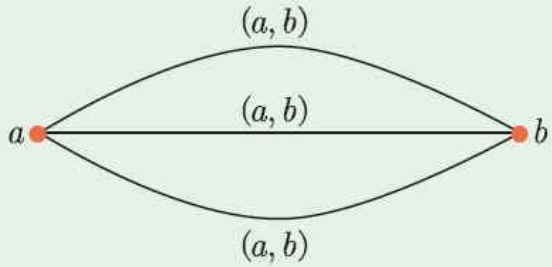
8.1.2 그래프의 형태

정의 8-2 **단순그래프(simple graph)**

그래프 $G=(V,E)$에서 임의의 두 정점 사이에 오직 하나의 변이 있는 그래프

정의 8-3 **다중그래프(multi-graph)**

그래프 $G=(V,E)$에서 임의의 두 정점 사이에 두 개 이상의 변이 있는 그래프



- $G=(V,E)$
- $V=\{a,b\}$
- $E=\{(a,b),(a,b),(a,b)\}$

그래프의 종류

- 무향 그래프
- 유향 그래프

무향 그래프(undirected graph)

- 그래프 G에서 정점들의 쌍인 간선 E를 생각해보자.
- $E_1 = (v_1, v_2)$, $E_2 = (v_2, v_1)$라 하면 E_1과 E_2는 두 개의 정점 v_1과 v_2를 연결하는 간선이다. (여기서 v_1, v_2는 두 개의 서로 다른 정점이다).
- 두 개의 정점의 순서를 생각하지 않는다면 E_1과 E_2는 같은 간선이 되고 두 개의 정점의 순서를 생각한다면 E_1과 E_2는 다른 간선이 된다.
- 정점의 순서를 무시하는 그래프 $G = (V, E)$를 무향 그래프 혹은 그래프라 하고, 정점의 순서를 생각하는 그래프를 유향 그래프라 한다.

유향 그래프(directed graph)

정의 7-3 유향 그래프

유향 그래프directed graph; digraph는 순서쌍 $G=(V, E)$이다. V는 정점의 집합이고 E는 V에 관한 이항 관계이며, 이는 간선의 집합이 된다.

❖ 시점(initial point; source), 종점(terminal point; target)

- 유향 그래프의 정점들의 순서쌍을 (v_1, v_2)라 할 때
  v_1은 시점이라 하고 v_2는 종점이라 하며,
  v_1에서 v_2로의 간선은 화살표($\rightarrow$)로 표시한다.

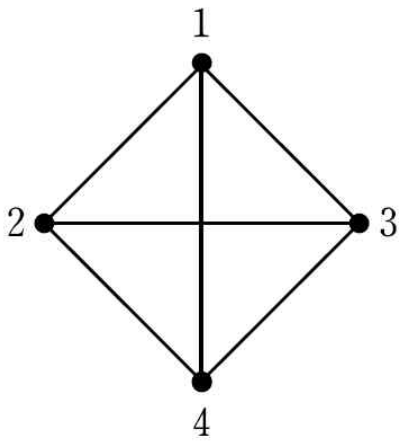
유향 그래프

예제 **무향 그래프와 유향 그래프 그리기**

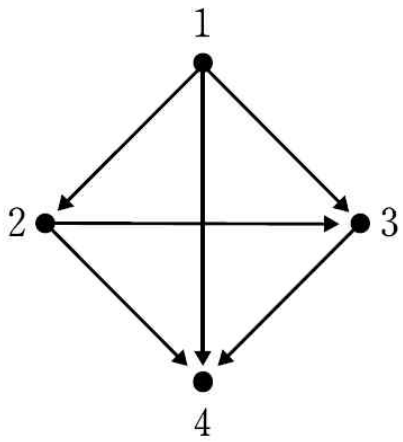
그래프 $G=(V,\ E)$를 무향 그래프와 유향 그래프로 나타내라. 단, $V=\{1,\ 2,\ 3,\ 4\}$, $E=\{(1,\ 2),\ (1,\ 3),\ (1,\ 4),\ (2,\ 3),\ (2,\ 4),\ (3,\ 4)\}$이다.

풀이

무향 그래프로 그리면 그림 (a)와 같고, 유향 그래프로 그리면 그림 (b)와 같다.



(a)



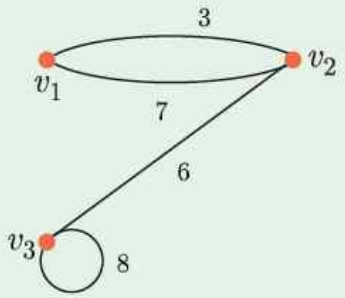
(b)

8.1.2 그래프의 형태

정의 8-5 **가중치그래프(weighted graph)**

그래프 $G=(V,E)$를 구성하는 각 변에 가중치가 부여된 그래프

- $W[v_i, v_j]$: 두 정점 v_i와 v_j에 근접하는 변에 부여된 가중치 표기



- $W[v_1, v_2] = 3,\ W[v_1, v_2] = 7,\ W[v_2, v_3] = 6,\ W[v_3, v_3] = 8$

8.1.3 그래프의 차수

정의 8-6 **차수(degree : $d(v)$)**

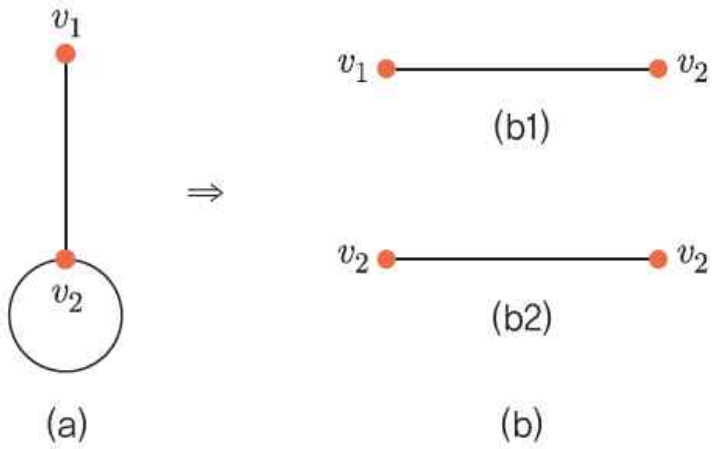
정점 v에 근접하는 변의 개수

방향그래프에 대하여,

(a) **외차수**(out-degree : $d_{out}(v)$) : 정점 v를 시작점으로 하는 화살표의 개수 / 진출 차수

(b) **내차수**(in-degree : $d_{in}(v)$) : 정점 v를 끝점으로 하는 화살표의 개수 / 진입 차수

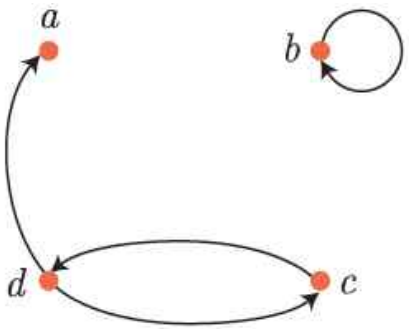
8.1.3 그래프의 차수



[그림 8-3] 그래프의 차수

- **(b1)** : $d(v_1) = 1,\ \ d(v_2) = 1$
- **(b2)** : $d(v_2) = 1,\ \ d(v_2) = 1$

$\therefore d(v_1) = 1,\ \ d(v_2) = 3$



[그림 8-4] 방향그래프의 차수

- $d_{out}(a) = 0,\ \ d_{in}(a) = 1$
- $d_{out}(b) = 1,\ \ d_{in}(b) = 1$
- $d_{out}(c) = 1,\ \ d_{in}(c) = 1$
- $d_{out}(d) = 2,\ \ d_{in}(d) = 1$

8.1.3 그래프의 차수

정리 8-1 **차수에 대한 정리**

(a) 그래프 $G=(V, E)$에서 모든 정점의 차수의 합은 변의 개수의 2배이다.

$$\sum_{v \in V} d(v) = 2|E|$$

(b) 그래프 $G=(V, E)$에서 차수가 홀수인 정점의 개수는 짝수이다.

증명

(a) 그래프에서 변 e는 항상 2개의 정점 u, v에 근접한다. 그러므로 변 e는 정점 u의 차수를 구할 때와 정점 v의 차수를 구할 때 두 번 계산된다.

$\therefore$ 하나의 변은 정점 2개의 차수를 계산하는 데 항상 중복되므로 모든 정점의 차수의 합은 변의 개수의 2배이다.

8.1.3 그래프의 차수

(b) 그래프 $G=(V,E)$에서 차수가 짝수인 정점의 집합이 V_e, 차수가 홀수인 정점의 집합을 V_o일 때, (a)에 의해 그래프의 변의 개수는 다음과 같이 구할 수 있다.

$$2|E| = \sum_{v \in V} d(v) = \sum_{v \in V_e} d(v) + \sum_{v \in V_o} d(v) \quad \cdots \ (*)$$

이때 각 $v \in V_e$에 대하여 $d(v)$가 짝수이므로 $\sum_{v \in V_e} d(v)$는 짝수이다.

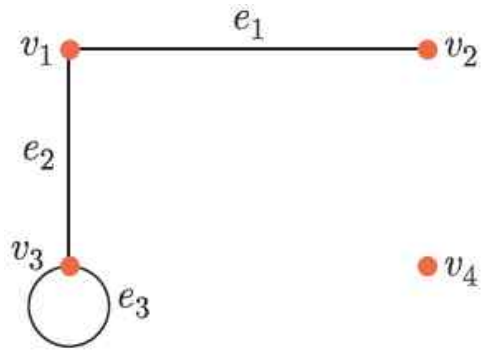
$$\sum_{v \in V_e} d(v) = 2k \ (k \in \mathbb{N}) \quad \cdots \ (**)$$

식 (**)를 이용하여 식 (*)를 다시 작성하면 다음과 같다.

$$\begin{aligned} 2|E| &= \sum_{v \in V} d(v) = \sum_{v \in V_e} d(v) + \sum_{v \in V_o} d(v) \\ &= 2k + \sum_{v \in V_o} d(v) \end{aligned}$$

따라서 $\sum_{v \in V_o} d(v) = 2|E| - 2k = 2(|E| - k)$이므로 $\sum_{v \in V_o} d(v)$도 짝수이다. 그런데 각 $v \in V_o$에 대하여 $d(v)$가 홀수이므로 $\sum_{v \in V_o} d(v)$가 짝수이려면 집합 V_o의 기수($|V_0|$), 즉 집합 V_o에 포함된 정점의 개수가 짝수여야 한다. 그러므로 차수가 홀수인 정점의 개수는 짝수이다. ■

8.1.3 그래프의 차수



- $d(v_1) = 2, \quad d(v_2) = 1, \quad d(v_3) = 3, \quad d(v_4) = 0$
- $|E| = 3$
- $\sum_{v \in V} d(v) = d(v_1) + d(v_2) + d(v_3) + d(v_4) = 2 + 1 + 3 + 0$

$$= 2 \times 3 = 6 = 2|E|$$

8.2 그래프의 종류

완전 그래프

정의 완전 그래프

무향 그래프 $G=(V, E)$의 모든 정점의 쌍 사이에 간선이 존재하면 무향 그래프 G를 **완전 그래프**complete graph라 한다.

❖ 완전 그래프의 간선 개수

- 각 꼭지점이 다른 모든 꼭지점들과 연결되는 그래프
- n개의 정점을 갖는 완전 그래프 K_n 는 $\frac{n(n-1)}{2}$개의 간선을 갖는다.

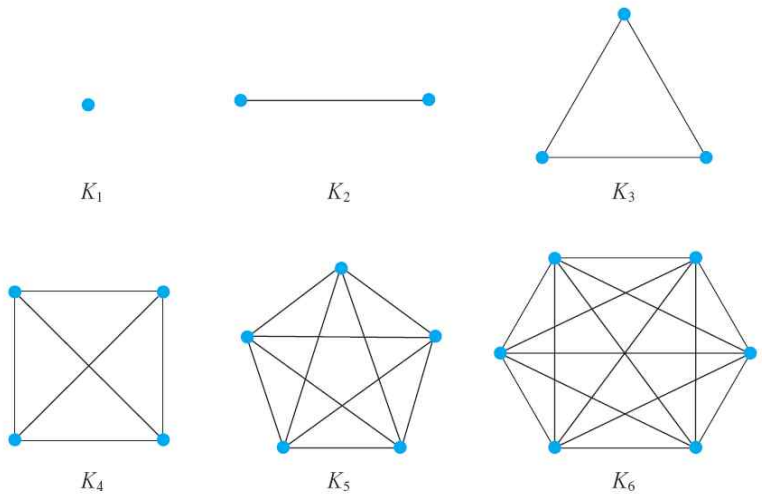
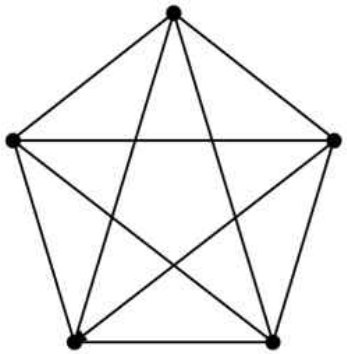


그림 출처: 부교재 이산수학 express,김대수 저

완전 그래프

예제 완전 그래프의 간선의 수

다음과 같은 완전 그래프가 있다. 간선의 수를 구하라.



풀이

정점의 개수가 5개인 완전 그래프이다. 그러므로 간선의 개수는 $\frac{5(5-1)}{2}=10$개이다.

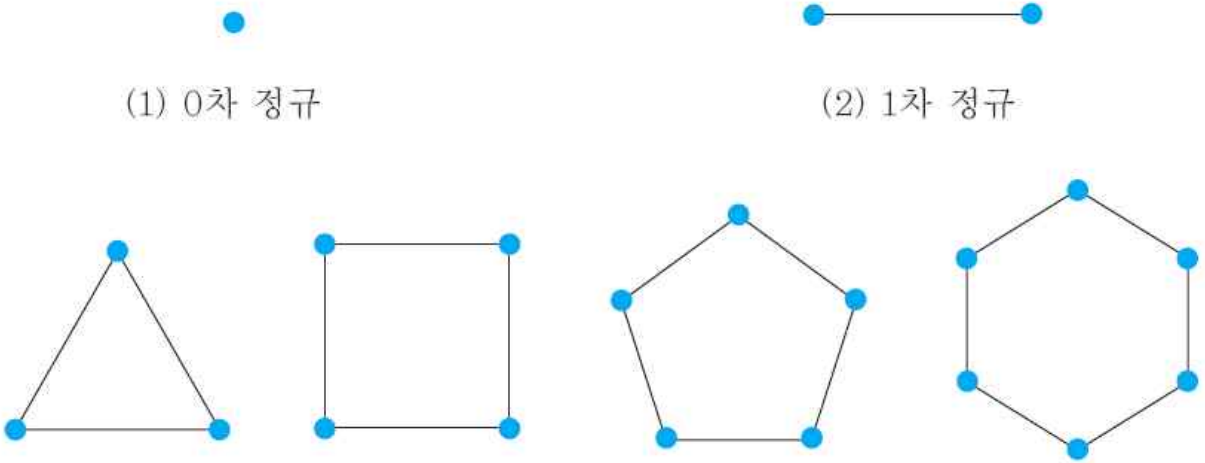
- [예제]과 같은 그래프는 5개의 정점을 가진 완전 그래프이다.
- 그러므로 각 정점에서 차수를 구해보면 4차이다.
- n개의 정점을 갖는 완전 그래프는 모든 정점들이 $(n-1)$ 차수를 가진다.
- 그래서 이런 그래프를 4차의 완전 그래프라 한다.

정규 그래프

정의 정규 그래프

그래프 $G=(V, E)$의 모든 정점의 차수가 같으면 그래프 G를 **정규 그래프**regular graph라 한다.

- 모든 꼭지점의 차수가 k이면 G를 k 차 정규그래프라고 한다.



(1) 0차 정규 (2) 1차 정규

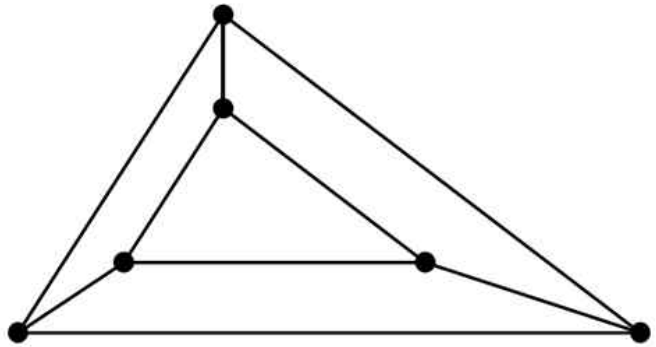
(3) 2차 정규

그림 출처: 부교재 이산수학 express,김대수 저

정규 그래프

예제 **정규 그래프**

다음의 그래프가 정규 그래프인지를 판단하라.



풀이

정규 그래프인지를 판단하기 위해서는 각 정점의 차수를 계산해야 한다. 이 그래프에서 모든 정점의 차수는 3이므로 3차의 정규 그래프이다.

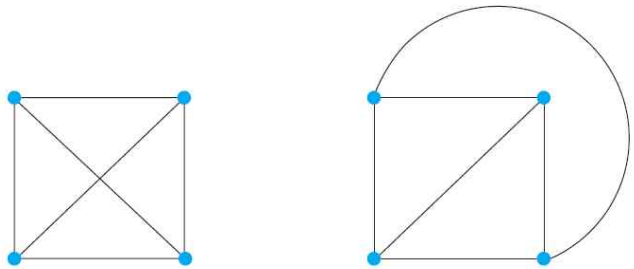
동형 그래프, 준 동형 그래프

정의 동형 그래프, 준동형 그래프

그래프 $G=(V, E)$와 $G^*=(V^*, E^*)$에서 함수 $f: V \rightarrow V^*$가 $(u, v) \in E \Leftrightarrow (f(u), f(v)) \in E^*$를 만족하는 전단사 함수이면, 이때 G와 G^*를 **동형 그래프**isomorphic graph라 한다. 또한 임의의 그래프 G에서 G의 간선 위에 정점을 추가해서 새로운 그래프를 만들 수 있다. G'과 G''이 그래프 G의 간선 위에 정점을 추가해서 만들어진 그래프라 하면, 이때의 G', G''을 그래프 G의 **준동형 그래프**homomorphism graph라 한다.

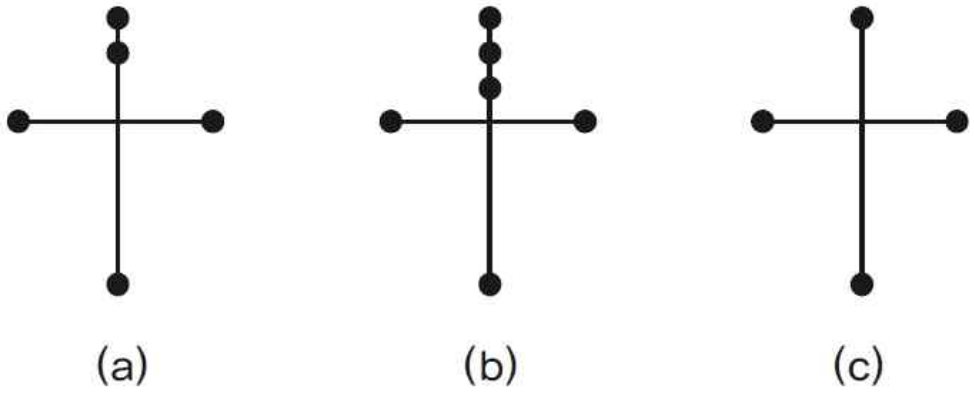
⇔ 그래프의 생김새가 다르더라도 동일한 정점의 집합과 연결선의 집합으로 구성된 그래프

⇔ 두 그래프의 간선 내의 점들을 1:1 대응 할 수 있는 관계 함수를 찾아줄 수 있어야 함



〈그림 7.12〉 동형 그래프의 예

동형 그래프, 준 동형 그래프



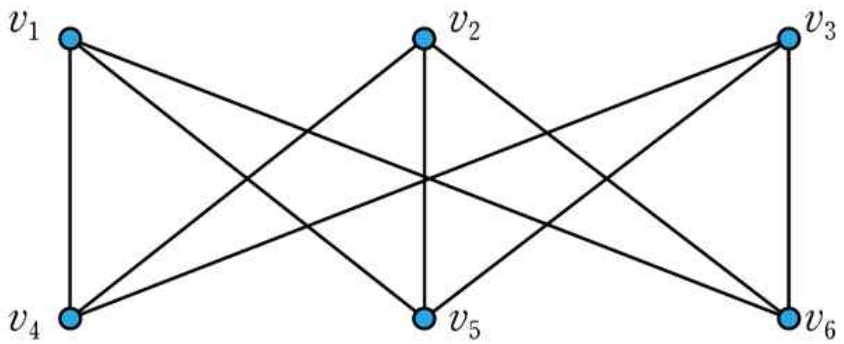
[그림 7-6] 준동형 그래프

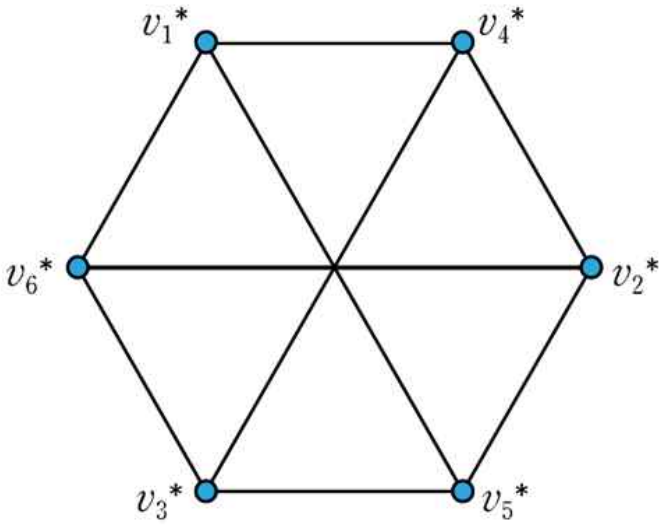
- [그림 7-6]의 그래프 (a)와 (b)는 (c)의 준 동형 그래프이다.

동형 그래프, 준 동형 그래프

예제 **동형 그래프 판단**

다음 두 그래프가 동형 그래프인지를 판단하라.





풀이

$V=\{v_1, v_2, v_3, v_4, v_5, v_6\}$을 $V^*=\{v_1^*, v_2^*, v_3^*, v_4^*, v_5^*, v_6^*\}$에 대응시키면 $(u, v) \in E \Leftrightarrow (f(u), f(v)) \in E^*$를 모두 만족한다. 그러므로 이 두 그래프는 동형 그래프이다.

부분 그래프, 스패닝 그래프

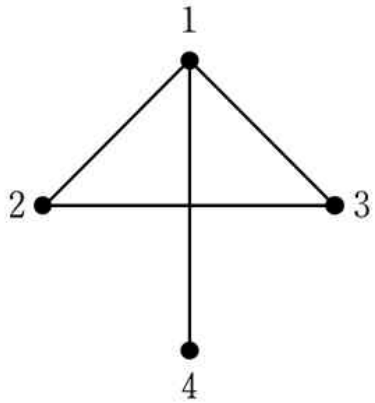
정의 부분 그래프, 스패닝 그래프

그래프 $G=(V, E)$가 있을 때 $V' \subseteq V$이고 $E' \subseteq E$인 그래프 $G'=(V', E')$을 G의 **부분 그래프**subgraph라고 한다. 특히 $E' \neq E$이면 **진부분 그래프**proper subgraph라고 한다. 그리고 $V'=V$이고 $E' \subseteq E$이면 G'은 G의 **스패닝 그래프**spanning graph라고 한다.

부분 그래프, 스패닝 그래프

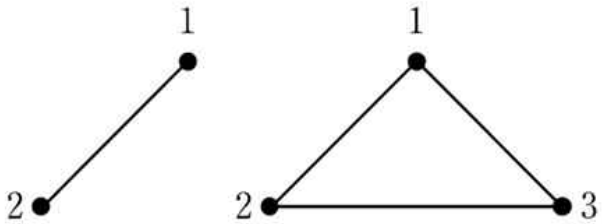
예제 **부분 그래프**

다음 그래프의 부분 그래프를 그려라.

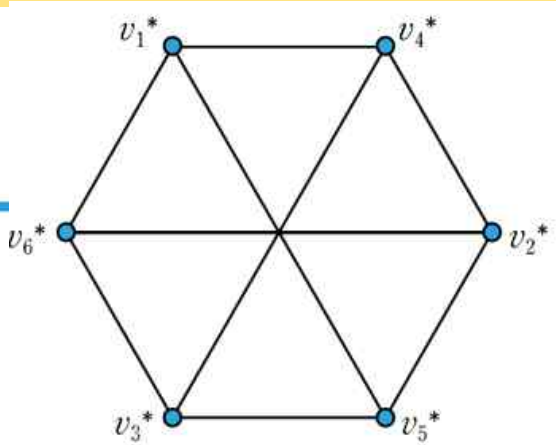


풀이

부분 그래프를 그리는 방법은 여러 가지가 있다. 그 중에서 두 개를 그리면 다음과 같다.



부분 그래프, 스패닝 그래프

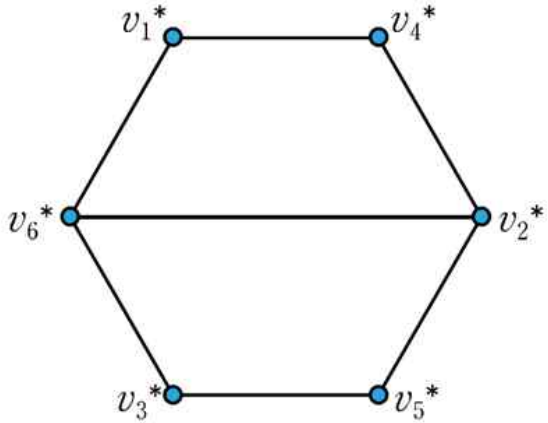


예제 스패닝 그래프

[예제 7-10]의 그래프에 대한 스패닝 그래프를 그려라.

풀이

스패닝 그래프를 그리는 방법은 여러 가지가 있다. 그 중에서 하나를 그리면 다음과 같다.

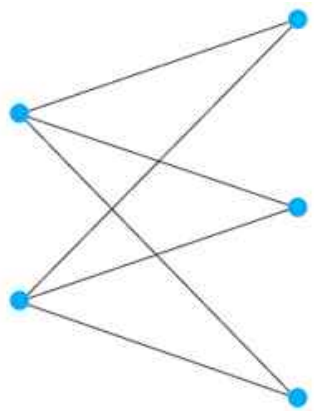


이분 그래프, 완전 이분 그래프

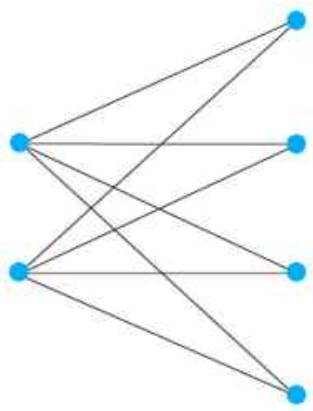
정의 이분 그래프, 완전 이분 그래프

그래프 $G=(V, E)$에서 정점의 집합 V가 두 부분집합 M과 N으로 분할되어($V=M\cup N$, $M\cap N=\varnothing$), 두 개의 집합 M과 N 사이에 간선으로 정점들을 연결하는 그래프 G를 **이분 그래프**bipartite graph라 한다. 특히 M 내의 모든 정점과 N 내의 모든 정점 사이에 모두 간선이 존재하면 G를 **완전 이분 그래프**complete bipartite graph라 한다. 즉, 완전 그래프이면서 이분 그래프를 만족한다.

- 그래프 $G=(V, E)$에서 정점의 집합 V가 두 부분집합 M과 N으로 분할되어 있고, M의 정점의 개수가 m이고 N의 정점의 개수가 n인 완전 이분 그래프를 $K_{m,n}$으로 나타내며, $K_{m,n}$은 $m\times n$의 간선을 가진다.



(1) $K_{2,3}$



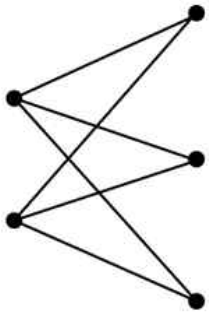
(2) $K_{2,4}$

이분 그래프, 완전 이분 그래프

예제 **완전 이분 그래프**

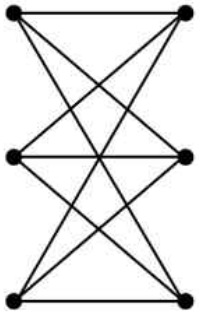
다음 그래프가 완전 이분 그래프인지 판단하라.

(a)



$K_{2,3}$

(b)



$K_{3,3}$

풀이

(a) 왼쪽에 있는 두 개의 정점과 오른쪽에 있는 세 개의 정점으로 분할하면 왼쪽과 오른쪽 간에 모든 정점들 사이에 간선이 존재하므로 완전 이분 그래프이다.

(b) 왼쪽에 있는 세 개의 정점과 오른쪽에 있는 세 개의 정점으로 분할하면 왼쪽과 오른쪽 간에 모든 정점들 사이에 간선이 존재하므로 완전 이분 그래프이다.

희소 그래프, 밀집 그래프

정의 희소 그래프, 밀집 그래프

그래프 $G=(V, E)$의 간선의 개수가 $|V|^2$보다 훨씬 작은 그래프를 **희소**sparse **그래프**라 하고 주로 인접 리스트를 사용해서 표현한다. **밀집**dense **그래프**는 간선의 개수가 $|V|^2$과 비례하는 그래프를 말하고, 이는 주로 인접 행렬을 사용해서 표현한다.

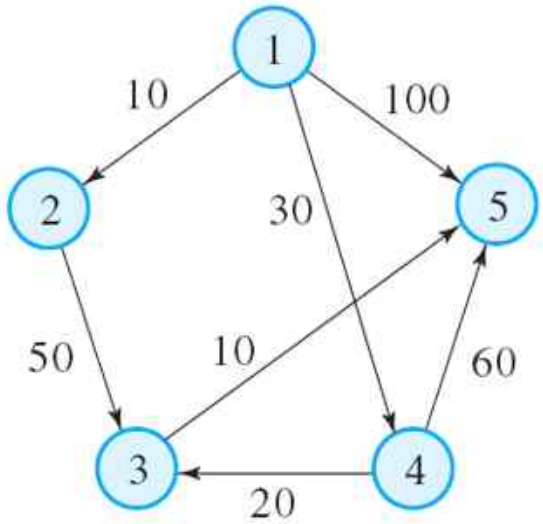
- 밀집그래프 : 간선(변)의 수가 최대 간선의 수에 가까운 그래프
- 희소 그래프와 밀집 그래프는 정점과 간선의 개수를 가지고 판단한다.
- 하지만 집과 희소 간의 구별은 다소 모호하므로 문맥에 따라 달라질 수 있다. 얼마를 기준으로 하느냐는 없다.

가중치 그래프

정의 가중치 그래프

가중치weighted **그래프**는 그래프 $G=(V, E)$의 간선에 음수가 아닌 가중치를 할당한 그래프이다.

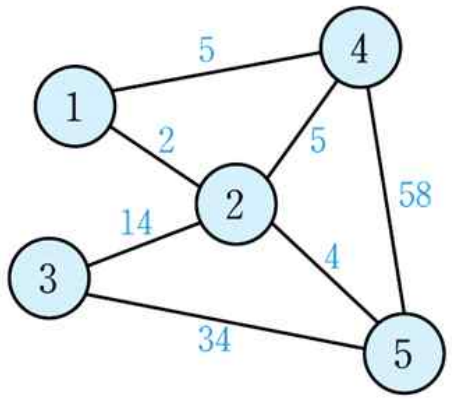
- 가중치는 두 정점 사이의 거리, 시간, 비용(cost) 등을 나타낸다.
- 가중치 그래프는 전력선 연결, 파이프라인 연결, 도로망 건설, 컴퓨터 네트워크 등에 응용된다.



가중치 그래프

예제 가중치 그래프

다음과 같은 가중치 그래프를 보고 정점 1로부터 정점 5로 가는 가장 작은 가중치를 가지는 경로를 구하라.



풀이

정점 1에서 정점 5로 가는 가장 작은 가중치를 가지는 경로는 1, 2, 5로서 가중치 6을 가진다.

트리

정의 트리

순환을 갖지 않고 루트 노드root node라고 하는 한 개의 특별한 정점을 갖는 연결 그래프를 **트리**tree 혹은 **수형도**라 한다.

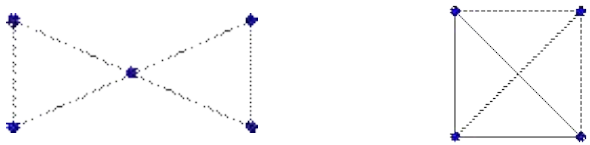
8.2.4 평면그래프

정의 8-12 **평면그래프(planar graph)**

연결그래프 $G=(V,E)$를 평면에 그릴 때 정점이 아닌 위치에서는 어떤 변도 교차하지 않도록 그릴 수 있는 그래프

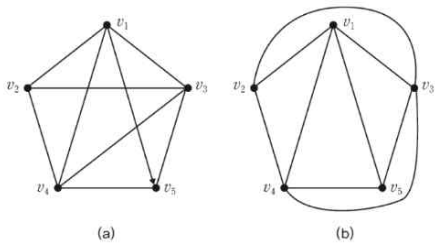
⇔ 정점과 정점을 잇는 모든 간선이 평면상에서 교차하지 않는 그래프
⇔ 평면 상에 그래프를 그렸을 때, 두 변이 꼭짓점 이외에 만나지 않도록 그릴 수 있는 그래프를 의미.

- (예제)



이 그림 상에서는 두 변이 만나지만, 만나지 않도록 그릴 수 있다

- (예제) [그림 7-11(a)] : 평면 그래프.
  - 왜냐하면 [그림 7-11(b)]와 같이 교점이 없이 그래프를 그릴 수 있기 때문이다.
- 평면 그래프를 평면상에 그린 것을 도형(map)이라 한다.



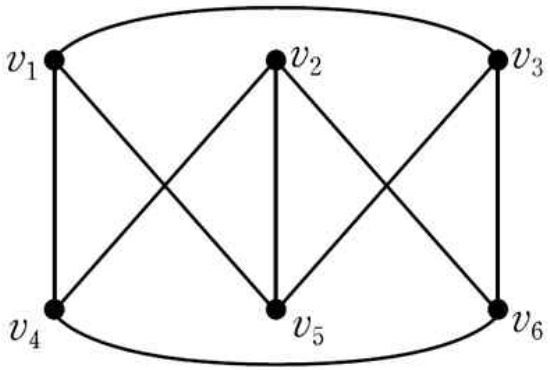
[그림 7-11] 평면 그래프

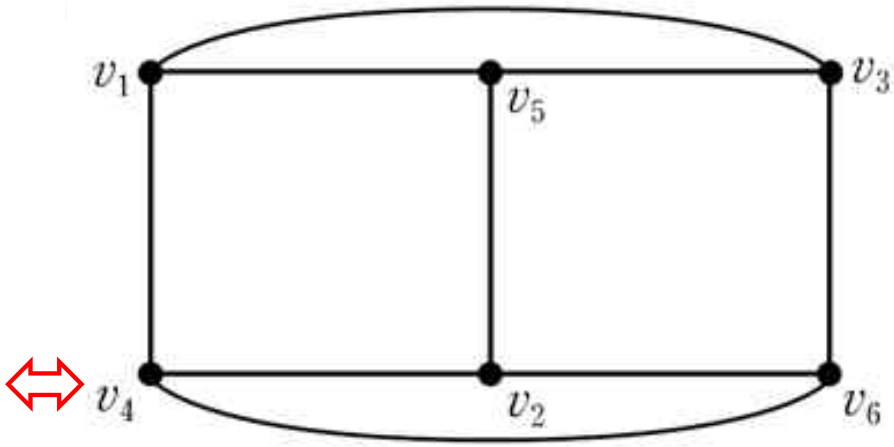
8.2.4 평면그래프

예제 평면 그래프 그리기

다음의 그래프를 평면 그래프로 바꿔라.

(a)



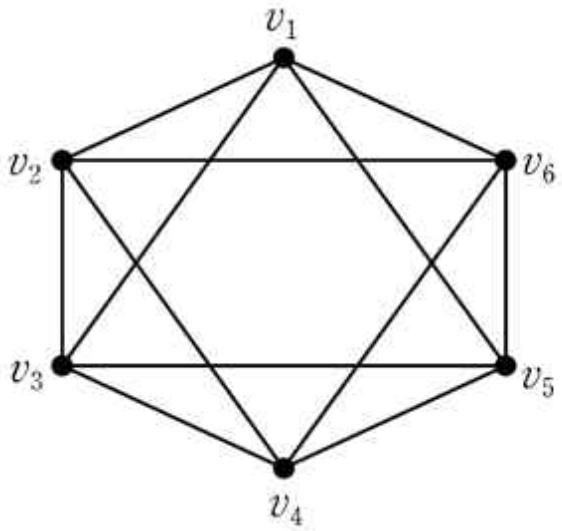


풀이

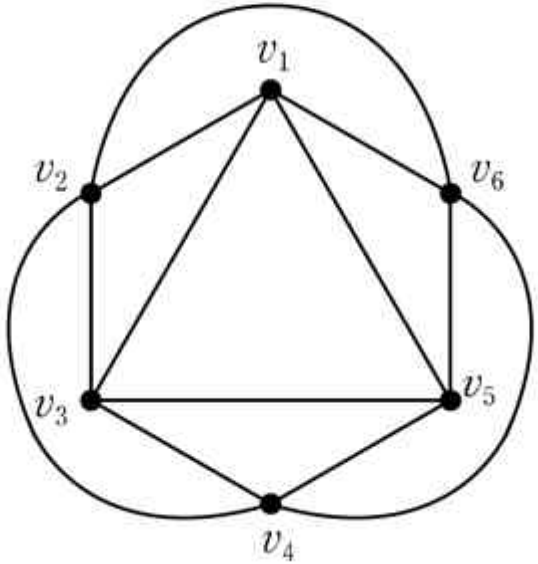
(a) v_2와 v_5의 위치만 바꾸기만 하면 된다.

8.2.4 평면그래프

(b)

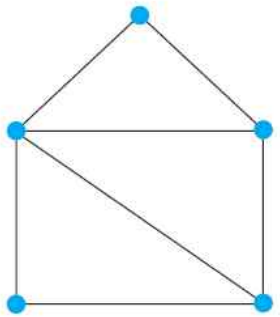


(b) v_2와 v_4를 연결하는 간선과 v_1과 v_5를 연결하는 간선을 다음과 같이 그리면 된다.

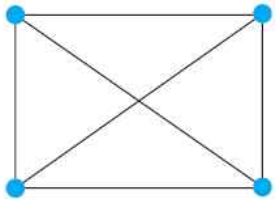


8.2.4 평면그래프

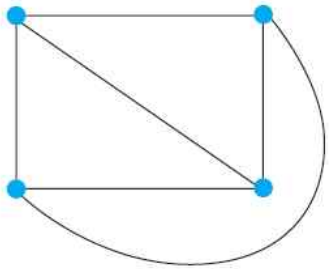
- (예제) 다음 그래프들이 평면 그래프인지의 여부를 판단하여라.



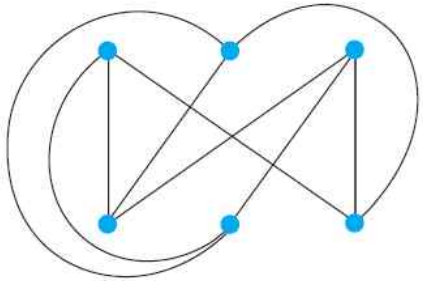
(1)



(2)



(3)



(4)

- (풀이)
  - ① (1) 간선들 서로 교차하지 않으므로 평면 그래프
  - ② (2) 와 (3)은 동형이므로 둘 다 평면 그래프
  - ③ (4) 평면 그래프가 아님

8.2.4 평면그래프

정리 8-2 **오일러 공식(Euler's formula)**

평면그래프 G에서 정점의 수를 v, 변의 수를 e, 영역의 수를 s라고 할 때, 다음 오일러 공식이 성립된다.

$$v-e+s=2$$

증명

오일러 공식을 증명하기 위해 변의 수를 이용한 수학적 귀납법을 이용한다.

❶ **기본 가정** : $e=1$일 때, 변이 하나인 그래프는 다음과 같이 두 종류이다.



그래프 A 그래프 B

그래프 A : 루프에 의해 닫힌 영역 하나와 그 바깥 영역 하나로 영역이 2개 존재한다.

$v=1,\ e=1,\ s=2 \Rightarrow v-e+s=1-1+2=2$이므로, 오일러 공식이 성립한다.

그래프 B : 열린 영역 1개가 존재한다.

$v=2,\ e=1,\ s=1 \Rightarrow v-e+s=2-1+1=2$이므로, 오일러 공식이 성립한다.

8.2.4 평면그래프

❷ **귀납 가정** : $e=k$일 때, $v-k+s=2$가 성립한다고 가정한다.

❸ **귀납 증명** : $e=k+1$일 때, 평면그래프에 변을 하나 추가하는 방법은 다음과 같이 두 가지 경우가 있다.

(i) 정점을 하나 추가하여 기존의 정점 하나와 연결하면 $e=k+1$이고 $v'=v+1$이다. 이때 영역의 수는 증가하지 않는다.

$$v'-e+s=(v+1)-(k+1)+s=v-k+s=2 \quad (\because \text{ 귀납 가정 } v-k+s=2)$$

(ii) 기존의 정점 사이에 변을 하나를 추가하면 $e=k+1$이다. 이때 정점의 수는 증가하지 않지만 영역의 수는 하나 증가하여 $s'=s+1$이다.

$$v-e+s'=v-(k+1)+(s+1)=v-k+s=2 \quad (\because \text{ 귀납 가정 } v-k+s=2)$$

$\therefore$ 오일러 공식이 성립한다. ■

8.3 그래프의 표현

그래프의 표현

그래프를 표현하는 방법

- 그래프는 그림을 이용하여 표현하는 것으로 가장 자연스럽고 이해하기에도 가장 쉬운 방법임
- 컴퓨터는 그림으로 표현된 정보를 이용할 수 없기 때문에 통상 인접 행렬이나 인접 리스트에 의해 표현됨
- 이를 통하여 컴퓨터 프로그램으로 구현됨

❖ **인접 행렬**

❖ **인접 리스트**

인접 행렬로 표현한 그래프

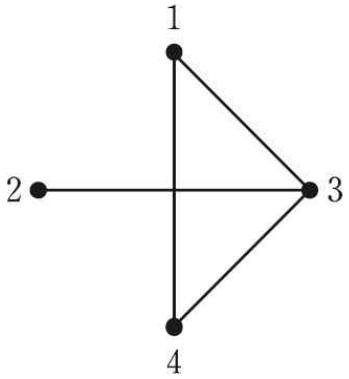
인접 행렬(adjancy matrix)로 표현한 그래프

- 인접 행렬이란 그래프의 연결 관계를 2차원 배열로 나타내는 방법이다.
  - 유향 그래프를 인접 행렬로 나타내고자 할 때 i번째 정점이 시점이고 j번째 정점이 종점인 간선이 있으면 인접 행렬 M의 i행 j열인 $M[i][j]$를 1로 하고, 없으면 $M[i][j]$를 0으로 하면 된다.
  - 무향 그래프를 인접 행렬로 나타내고자 할 때 i번째 정점과 j번째 정점을 연결하는 간선이 존재하면 $M[i][j]$와 $M[j][i]$를 1로 하고 존재하지 않으면 0으로 하면 된다.
  - [그림 7-3]은 이러한 예를 보인 것이다. 그리고 $M[i][j]$가 1이라면 정점 v_i와 v_j는 인접 정점(adjacent vertex)이라 한다.
- 그래프 G = (V, E)에서 |V| = n일 때 G의 인접 행렬은 n × n 행렬 M로 나타내어지며, 이때 M의 각 원소 a_{ij} 는 다음과 같이 정의됨

$$a_{ij} = \begin{cases} 1 & (v_i, v_j) \in E \\ 0 & \text{otherwise} \end{cases}$$

인접 행렬로 표현한 그래프

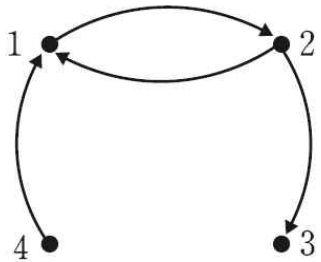
인접 행렬(adjancy matrix)로 표현한 그래프



(a) 무향 그래프

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \end{array}$$

(b) 인접 행렬



(c) 유향 그래프

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

(d) 인접 행렬

[그림 7-3] 인접 행렬로 표시한 그래프

인접 행렬로 표현한 그래프

인접 행렬(adjancy matrix)로 표현한 그래프

- 만약 가중치가 있는 그래프라면 인접 행렬의 값에 1 대신에 가중치의 값을 직접 넣어주면 된다.
- 무향 그래프에 대한 인접 행렬은 대칭 행렬이다.
- 장점은 간선의 수가 많을수록 매우 적합하고, 정점의 번호가 주어지면 배열의 접근만으로 바로 찾을 수 있다.
- 반면에, 단점은 간선의 수가 작다면 적합하지 않고 항상 $O(|V|2)$ 만큼의 기억 공간을 사용한다.

연결 리스트로 표현한 그래프

인접 리스트(linked list)로 표현한 그래프

- 각 정점에 대해 포인터(pointer)가 주어지고, 그 점으로부터 연결된 정점들을 차례로 연결 리스트(linked list)로 표시한다.
  - 연결 리스트: 각 노드가 데이터와 포인터를 가지고 한 줄로 연결되어 있는 방식으로 데이터를 저장하는 자료 구조
  - 그래프의 각 정점을 그 정점을 표시하는 레이블(label)과 다른 정점들을 가리키는 포인터(pointer)들의 리스트로 구성
  - 각 포인터는 그래프에서의 간선을 의미.
- 그래프 G를 구성하는 모든 정점에 대해 헤드 포인터를 부여하고, 연결 선이 있는 모든 정점을 헤드 포인터로부터 연결시켜 표현하는 방법

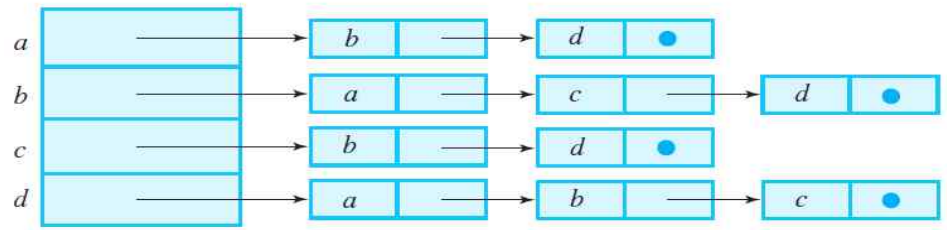
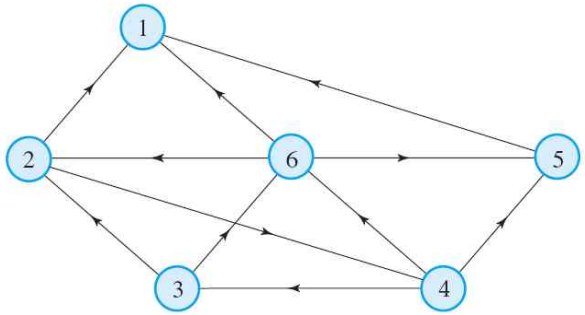


그림 출처: 부교재 이산수학 express,김대수 저

연결 리스트로 표현한 그래프

인접 리스트(linked list)로 표현한 그래프

- 예) 다음의 방향 그래프에서 인접 리스트를 구해보아라



- 풀이) 이 그래프에서는 정점 1에서 나가는 연결선이 없다.

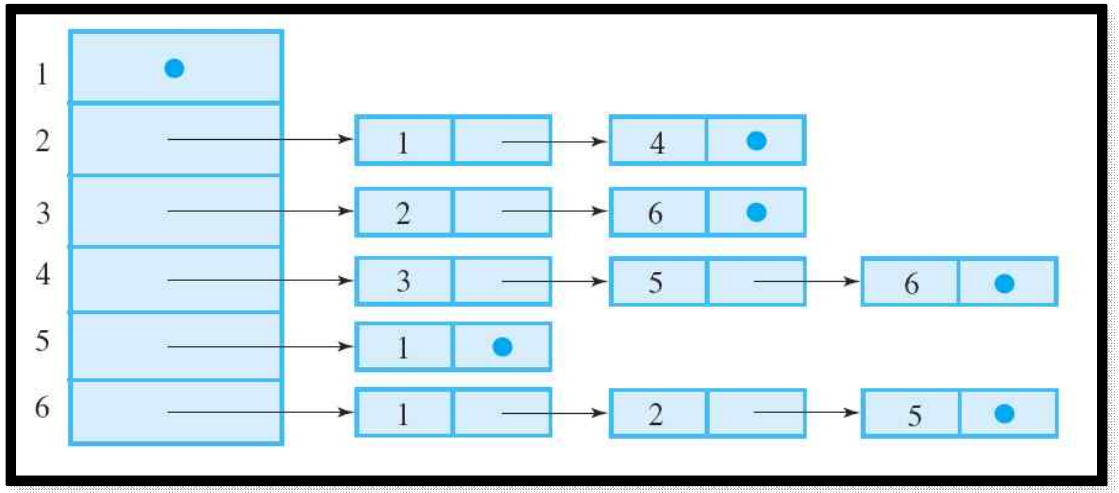


그림 출처: 부교재 이산수학 express,김대수 저

연결 리스트로 표현한 그래프

연결 리스트(linked list)로 표현한 그래프

- 장점은 인접 리스트는 $|V|$개의 연결 리스트에 실제 간선의 수만큼 원소가 들어있으므로 전체 $O(|V| + |E|)$의 기억 공간을 사용한다.

- 단점은 어떤 정점을 찾을 때 처음부터 검색해야 한다.

8.4 오일러와 해밀턴

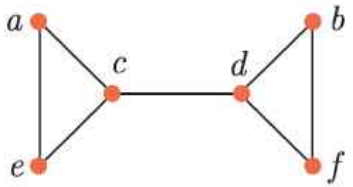
8.4 오일러와 해밀턴

정의 8-19 **순환(cycle) / 회로(circuit)**

연결그래프에서 시작하는 정점과 끝나는 정점이 같은 경로

정의 8-20 **길이(length)**

경로 또는 순환을 구성하는 변의 수



[그림 8-16] 길, 경로, 순환의 예

(1) $a-c-d-f$: $(a,c),(c,d),(d,f)$: 변 3개

(2) $a-e-c-d-b-f$: $(a,e),(e,c),(c,d),(d,b),(b,f)$: 변 5개

(3) $a-c-e-a$: $(a,c),(c,e),(e,a)$: 변 3개

(4) $a-e-c-a$: $(a,e),(e,c),(c,a)$: 변 3개

(5) $a-c-d-b-f-d-c-a$

8.4 오일러와 해밀턴

예제 8-16

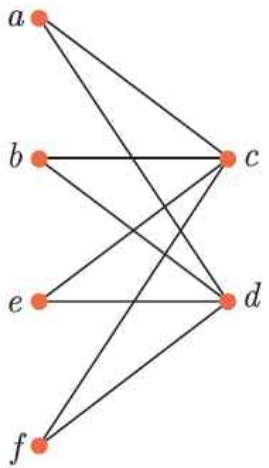
주어진 그래프를 보고 다음 물음에 답하라.

(a) 정점 a에서 정점 f까지의 가능한 경로 중 길이가 가장 짧은 경로를 모두 찾고 그 길이를 구하라.

(b) 정점 a에서 시작하는 가능한 순환 중 길이가 가장 짧은 순환을 모두 찾고 그 길이를 구하라.

(c) 정점 a에서 정점 f까지의 가능한 경로 중 길이가 가장 긴 경로를 모두 찾고 그 길이를 구하라.

(d) 정점 a에서 시작하는 가능한 순환 중 길이가 가장 긴 순환을 모두 찾고 그 길이를 구하라.



풀이

(a) 정점 a에서 정점 f까지의 경로 중 길이가 가장 짧은 경로와 그 길이는 다음과 같다.

$$a-c-f \quad / \quad a-d-f \quad (\text{길이 : } 2)$$

(b) 정점 a에서 시작하는 순환 중 길이가 가장 짧은 순환과 그 길이는 다음과 같다.

$$a-c-f-d-a \quad / \quad a-d-f-c-a \quad (\text{길이 : } 4)$$

(c) 정점 a에서 정점 f까지의 경로 중 길이가 가장 긴 경로와 그 길이는 다음과 같다.

$$a-c-b-d-e-c-f \quad / \quad a-c-e-d-b-c-f$$
$$a-d-b-c-e-d-f \quad / \quad a-d-e-c-b-d-f \quad (\text{길이 : } 6)$$

(d) 정점 a에서 시작하는 순환 중 길이가 가장 긴 순환과 그 길이는 다음과 같다.

$$a-c-b-d-e-c-f-d-a \quad / \quad a-c-e-d-b-c-f-d-a$$
$$a-d-b-c-e-d-f-c-a \quad / \quad a-d-e-c-b-d-f-c-a \quad (\text{길이 : } 8)$$

8.4.1 오일러 그래프

정의 8-21 **오일러 경로(Eulerian path)**

연결그래프 $G=(V,E)$의 임의의 정점 v에서 시작하여 모든 변을 꼭 한 번씩만 지나는 경로

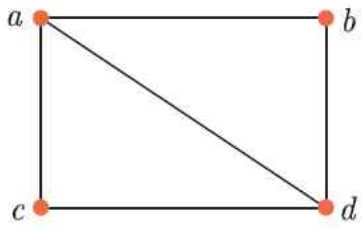
정의 8-22 **오일러 순환(Eulerian cycle) / 오일러 회로(Eulerian circuit)**

연결그래프 $G=(V,E)$의 임의의 정점 v에서 시작하여 모든 변을 꼭 한 번씩만 지나 다시 정점 v로 돌아오는 경로

- **오일러 그래프**(Eulerian graph) : 오일러 순환을 가지는 그래프

- 그래프 이론을 제시한 쾨니히스베르크의 다리 건너기 문제는 오일러 그래프에 대한 문제이다.
- 오일러 그래프는 한붓그리기로 잘 알려져 있다.
- 그래프의 한 정점에서 시작하여 붓을 떼지 않고 모든 간선을 딱 한 번씩만 지나서 다시 자기 자신으로 돌아오는 루프이다.

8.4.1 오일러 그래프



그래프 A

$a-b-d-c-a-d$ / $a-b-d-a-c-d$

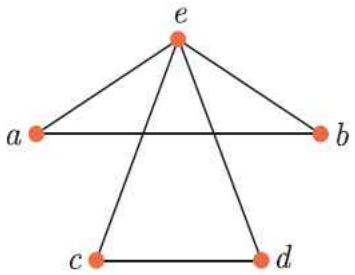
$a-d-b-a-c-d$ / $a-d-c-a-b-d$

$d-b-a-c-d-a$ / $d-b-a-d-c-a$

$a-c-d-b-a-d$ / $a-c-d-a-b-d$

$d-a-b-d-c-a$ / $d-a-c-d-b-a$

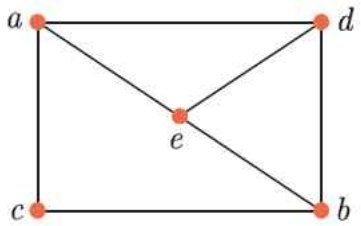
$d-c-a-b-d-a$ / $d-c-a-d-b-a$



그래프 B

$a-b-e-c-d-e-a$ / $a-b-e-d-c-e-a$

$a-e-c-d-e-b-a$ / $a-e-d-c-e-b-a$



그래프 C

8.4.1 오일러 그래프

정리 8-3 **오일러 그래프에 대한 정리**

(a) 연결그래프 $G=(V, E)$의 모든 정점의 차수가 짝수인 것과 오일러 그래프는 서로 필요충분조건이다.

(b) 연결그래프 $G=(V, E)$의 정점 중 차수가 홀수인 정점의 수가 0 또는 2개인 것과 그래프에 오일러 경로가 존재하는 것은 서로 필요충분조건이다.

8.4.1 오일러 그래프

증명

(a) 명제 : 연결그래프 $G=(V,E)$의 모든 정점의 차수가 짝수이다.

$\Leftrightarrow$ 연결그래프 $G=(V,E)$가 오일러 그래프이다.

(ⅰ) '연결그래프 $G=(V,E)$의 모든 정점의 차수가 짝수이다.

$\Rightarrow$ 연결그래프 $G=(V,E)$가 오일러 그래프이다.'를 보이자.

❶ $|V|=1$일 때, $v\in V$이고 $d(v)=2m(m\in\mathbb{N})$이므로 정점 v에 대한 루프가 m개 존재하고 정점 v에서 시작하는 오일러 순환이 존재한다.

$\therefore$ $|V|=1$일 때, 그래프 G는 오일러 그래프이다.

❷ $|V|=k$일 때, k개의 정점 모두의 차수가 $d(v_i)=2n(n\in\mathbb{N},1\le i\le k)$이면 그래프 G는 오일러 그래프라고 가정한다.

❸ $|V|=k+1$일 때, 귀납 가정에서 오일러 그래프라고 가정한 $|V|=k$개이고 각 정점의 차수가 $d(v_i)=2n(n\in\mathbb{N},1\le i\le k)$인 그래프 G를 편의상 G_1이라고 하자. 그래프 G_1에 $d(v)=2m$ $(m\in\mathbb{N})$인 정점 v 하나로 구성된 그래프 G_2를 연결할 때 각 정점의 차수는 $2p(p\in\mathbb{N})$여야 하므로 두 그래프 G_1과 G_2 사이에는 $2q(q\in\mathbb{N})$개의 변이 추가되어야 한다. 그러면 오일러 그래프 G_1과 그래프 G_2 사이에 순환이 만들어진다.

$\therefore$ $|V|=k+1$일 때, 그래프 G는 오일러 그래프이다.

$\therefore$ 연결그래프 $G=(V,E)$의 모든 정점의 차수가 짝수이면, 연결그래프 $G=(V,E)$는 오일러 그래프이다.

8.4.1 오일러 그래프

(ii) '연결그래프 $G=(V,E)$가 오일러 그래프이다.

$\Rightarrow$ 연결그래프 $G=(V,E)$의 모든 정점의 차수가 짝수이다.'를 보이자.

연결그래프 $G=(V,E)$가 오일러 그래프이므로 정점 $v\in V$에서 시작하여 정점 v로 끝나는 오일러 순환이 존재하고 이때 $d(v)=2m\,(m\in\mathbb{N})$이다. 또한 오일러 순환에 포함되는 각 정점은 정점으로 들어가는 변과 나오는 변을 갖게 되므로 각 정점의 차수는 짝수이다.

$\therefore$ 연결그래프 $G=(V,E)$가 오일러 그래프이면, 연결그래프 $G=(V,E)$의 모든 정점의 차수가 짝수이다.

따라서 명제 "연결그래프 $G=(V,E)$의 모든 정점의 차수가 짝수이다. $\Leftrightarrow$ 연결그래프 $G=(V,E)$는 오일러 그래프이다."는 참(T)이다.

8.4.1 오일러 그래프

(b) 명제 : 연결그래프 $G=(V,E)$의 정점 중 차수가 홀수인 정점의 수가 0 또는 2개이다.
$\Leftrightarrow$ 연결그래프 $G=(V,E)$가 오일러 경로를 갖는다.

(ⅰ) '연결그래프 $G=(V,E)$의 정점 중 차수가 홀수인 정점의 수가 0 또는 2개이다.
$\Rightarrow$ 연결그래프 $G=(V,E)$가 오일러 경로를 갖는다.'를 보이자.

$V=\{v_1, v_2, \cdots, v_k\}$일 때 모든 정점의 차수가 $2m\,(m\in\mathbb{N})$인 경우, 즉 홀수 차수인 정점이 0개인 경우에는 오일러 순환이 존재하여 시작하는 정점에서 끝나는 경로가 만들어진다. 차수가 $2n(n\in\mathbb{N})$인 정점 v_1과 v_2를 연결하는 변 (v_1, v_2) 중 하나를 제거하면 두 정점의 차수는 $2n-1$로 홀수가 되므로 그래프 G에서 정점 v_1에서 시작하여 정점 v_2로 끝나거나 정점 v_2에서 시작하여 정점 v_1으로 끝나는 경로가 만들어질 수 있다.

$\therefore$ 연결그래프 $G=(V,E)$의 정점 중 차수가 홀수인 정점의 수가 0 또는 2개이면, 연결그래프 $G=(V,E)$는 오일러 경로를 갖는다.

8.4.1 오일러 그래프

(ii) '연결그래프 $G=(V,E)$가 오일러 경로를 갖는다.
$\Rightarrow$ 연결그래프 $G=(V,E)$의 정점 중 차수가 홀수인 정점의 수가 0 또는 2개이다.'를 보이자.

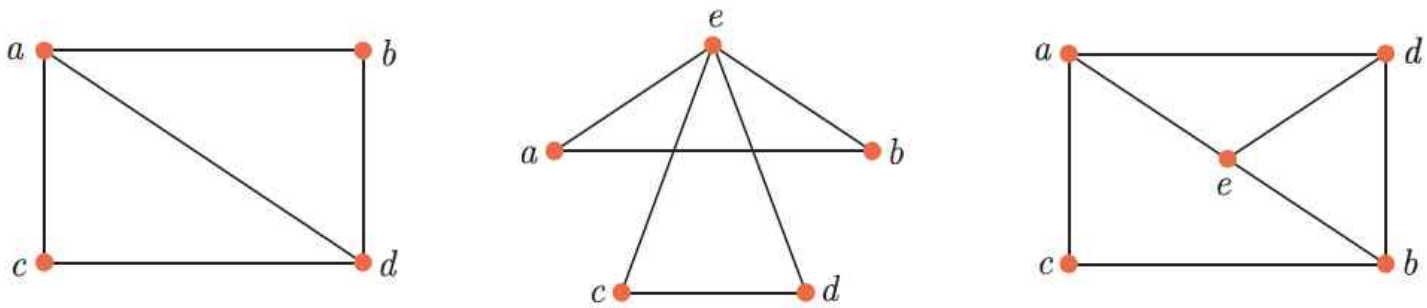
그래프 G가 오일러 경로를 가지므로 정점 v_1에서 시작하여 정점 v_2로 끝난다면 정점 v_1은 시작하는 변을, 정점 v_2는 끝나는 변을 각각 1개씩 기본적으로 가지므로, 두 정점 v_1, v_2의 차수는 모두 홀수이다. 그리고 두 정점 v_1, v_2를 제외한 나머지 정점은 하나의 변이 들어가고 다른 하나의 변이 나와 변이 정점을 통과하는 형태이므로 차수가 짝수이다. 그러므로 차수가 홀수인 정점은 v_1, v_2 2개뿐이다.

또한 그래프 G가 오일러 경로를 가지므로 정점 v_i에서 시작하여 다시 정점 v_i로 끝나는 경로가 만들어질 수도 있다. 이때 정점 v_i에서 다른 정점으로 나가는 변과 다른 정점으로부터 들어오는 변, 이 2개의 변을 기본적으로 가지므로 정점 v_i의 차수는 짝수이다. 또한 정점 v_i를 제외한 나머지 정점도 모두 들어가는 변과 나가는 변을 가져야 하므로 차수가 짝수이다. 따라서 그래프 G를 구성하는 모든 정점의 차수는 짝수이므로 차수가 홀수인 정점은 0개이다.

$\therefore$ 연결그래프 $G=(V,E)$가 오일러 경로를 가지면, 연결그래프 $G=(V,E)$의 정점 중 차수가 홀수인 정점의 수가 0 또는 2개이다.

따라서 명제 "연결그래프 $G=(V,E)$의 정점 중 차수가 홀수인 정점의 수가 0 또는 2개이다. $\Leftrightarrow$ 연결그래프 $G=(V,E)$가 오일러 경로를 갖는다."는 참(T)이다. ■

8.4.1 오일러 그래프



그래프 A 그래프 B 그래프 C

[그림 8-17] 오일러 경로, 오일러 순환이 있는 예와 없는 예

그래프 A : $d(a)=3,\ d(b)=2,\ d(c)=2,\ d(d)=3$

그래프 B : $d(a)=2,\ d(b)=2,\ d(c)=2,\ d(d)=2,\ d(e)=4$

그래프 C : $d(a)=3,\ d(b)=3,\ d(c)=2,\ d(d)=3,\ d(e)=3$

8.4.2 해밀턴 그래프

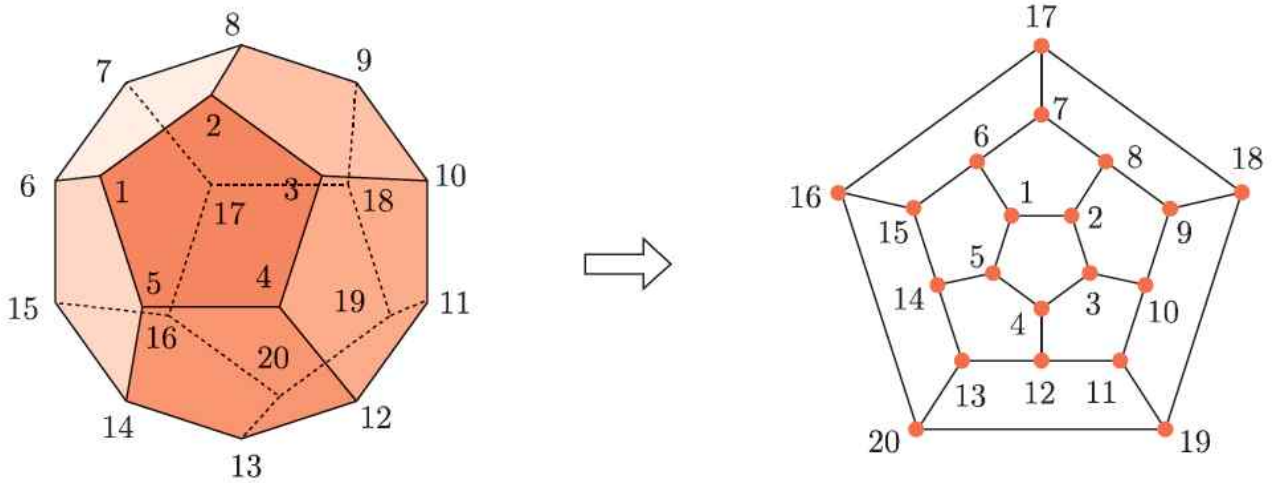
정의 8-23 **해밀턴 경로(Hamiltonian path)**

연결그래프 $G=(V, E)$의 모든 정점을 한 번씩만 지나는 경로

정의 8-24 **해밀턴 순환(Hamiltonian cycle) / 해밀턴 회로(Hamiltonian circuit)**

연결그래프 $G=(V, E)$의 임의의 정점 v에서 시작하여 모든 정점을 딱 한 번씩만 지나 정점 v로 다시 돌아오는 경로

- **해밀턴 그래프**(Hamiltonian graph) : 해밀턴 순환을 갖는 그래프

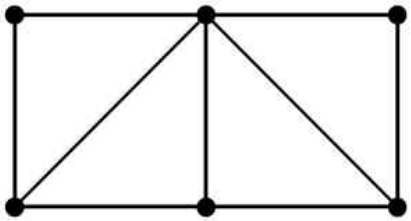


[그림 8-18] 12면체를 표현한 그래프

8.4.2 해밀턴 그래프

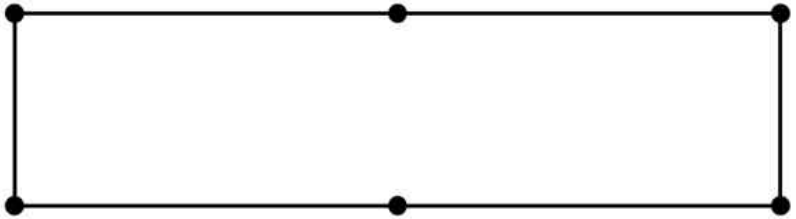
예제 오일러 그래프와 해밀턴 그래프 판단

다음 그래프가 오일러 그래프인지 해밀턴 그래프인지를 판단하라.



풀이

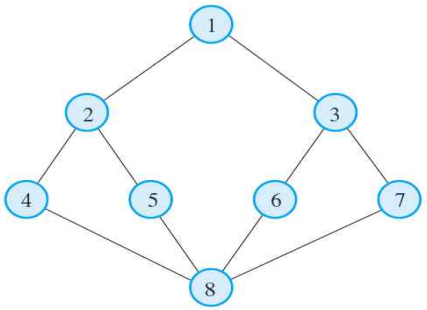
오일러 그래프가 되기 위해서는 모든 정점들의 차수가 짝수여야 한다. 그런데 모든 정점의 차수가 짝수가 아니므로 오일러 그래프는 아니다. 주어진 그래프는 다음과 같은 경로를 따라서 모든 정점을 지나 다시 시작점으로 올 수 있다. 그러므로 해밀턴 그래프이다.



즉, 해밀턴 그래프이지만 오일러 그래프는 아니다.

8.5 그래프의 활용

그래프의 탐색



그래프의 탐색

- 그래프의 각 정점을 방문하는 것을 탐색(traversal)이라고 한다.
  - 그래프에서 중요한 연산 중 하나
  - 무향 그래프 $G = (V, E)$와 V에 있는 정점 v가 주어져 있을 때,
    이 정점 v로부터 시작하여 도달할 수 있는 G의 모든 정점을 탐색하는 것
  - 많은 문제들이 단순히 그래프의 노드를 탐색하는 것으로 해결
    - (예) 도로망에서 특정 도시에서 다른 도시로 갈 수 있는지 여부
    - (예) 전자회로에서 특정 단자와 다른 단자가 서로 연결되어 있는지 여부

- **그래프의 각 정점을 한번씩만 방문하는 방법**
  - 깊이 우선 탐색(depth first search ; DFS)
  - 너비우선 탐색(breadth first search ; BFS)
  - 따라서, 두 방법 모두 각각의 노드에 방문 했는지 안했는지를 체크하는 check 배열이 필요.

깊이우선 탐색

깊이우선 탐색 (Depth First Search : DFS)

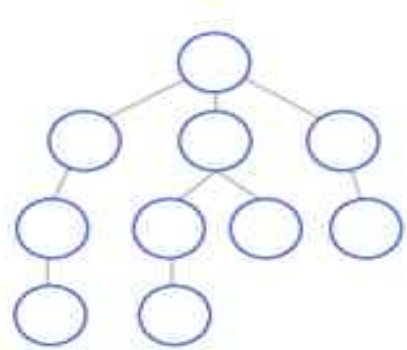
방법 : 자동차로 여행할 경우 방문지가 있으면 무조건 방문하는 방법이다. '최대한 깊숙히 많이 가는 방법'으로 모든 정점을 1번씩 방문하는 것이다. 즉 후진하지 않고 가는 방법이며 후진하는 경우는 길이 막히거나, 이미 방문했던 곳일 경우, 후진하여 방문할 곳은 마지막에 갈수 있었던 곳 중 가지 않았던 길이다. 이 방법을 깊이 우선 탐색이라고 하며 탐색 중 방문 가능한 곳은 스택 자료를 이용하여 저장해 둔다.

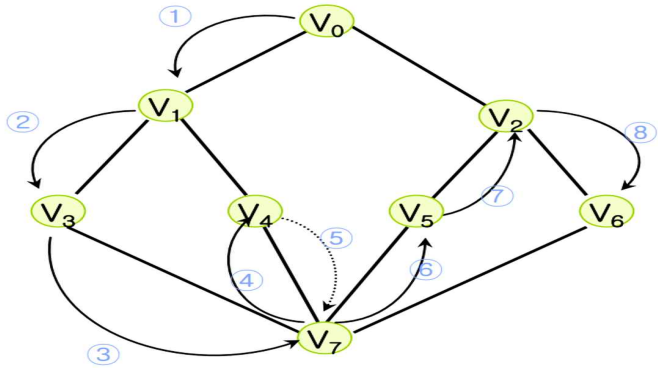
1. 먼저 주어진 정점 v를 출발점으로 하여 v 를 방문한다.
2. (우선적 탐색 기준에 따라) v에 인접하고 아직 방문하지 않은 정점 w를 선택하여 w를 출발점으로 해서 다시 깊이 우선 탐색을 시작한다.
3. 인접한 모든 정점들이 이미 방문한 정점 v인 경우는 가장 최근에 방문했던 (그 전에 마지막으로 방문했던) 정점 중에서, 방문하지 않은 정점 w를 가진 정점을 선택하여 정점 w로부터 다시 깊이우선 탐색을 시작한다.
4. 이것을 모든 정점이 한 번씩 방문 될 때까지 반복한다.

- 구현 : 스택 or 재귀호출

깊이우선 탐색

깊이우선 탐색 (Depth First Search : DFS)





우선적 탐색 기준

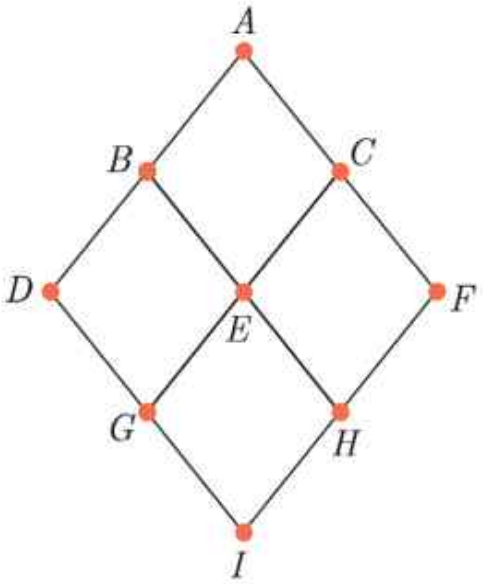
❶ 어떤 정점 v_n에 인접한 정점이 여러 개인 경우 탐색 기준에 따라 우선적으로 탐색하는 정점이 달라질 수 있다. 이 책에서는 가장 왼쪽에 있는 정점 또는 사전적 순서로 앞에 오는 정점부터 탐색한다.

❷ 어떤 정점 v_n에 인접한 모든 정점을 탐색하면 v_n에 인접하면서 v_n 직전에 탐색한 정점 v_{n-1}로 돌아가 v_{n-1}에 인접하면서 탐색하지 않은 다른 정점 v_{n+1}을 탐색한다. 이를 **백트래킹**(back tracking)이라고 한다.

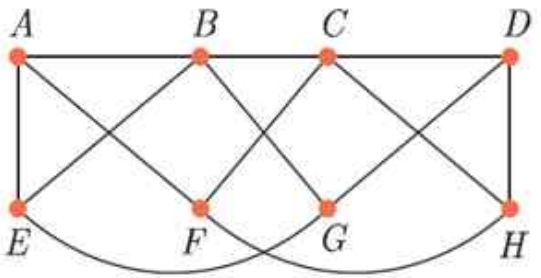
그래프의 탐색

다음 그래프에 대해 정점 A에서 시작하여 사전적 순서로 깊이 우선 탐색을 하고, 탐색 결과로 나오는 신장부분그래프를 그려라.

(a)

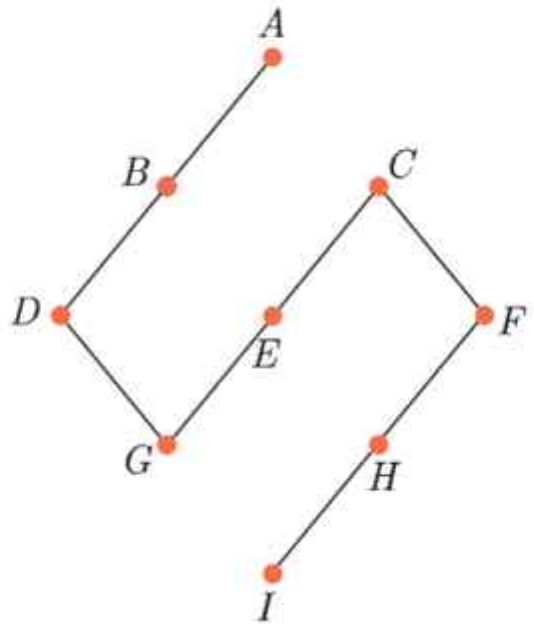


(b)

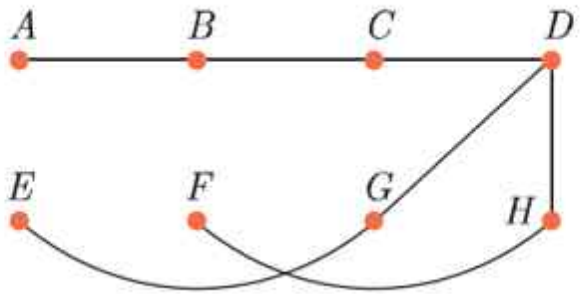


그래프의 탐색

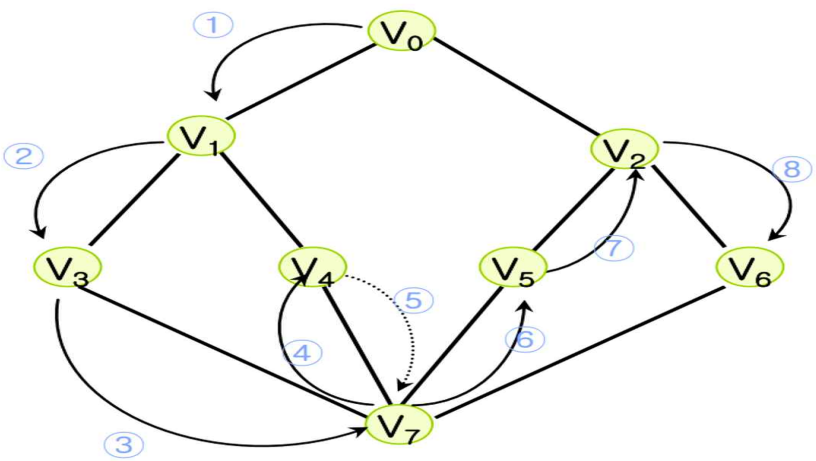
(a) $A-B-D-G-E-C-F-H-I$이고, 탐색 결과인 신장부분그래프는 다음과 같다.



(b) $A-B-C-D-G-E-H-F$이고, 탐색 결과인 신장부분그래프는 다음과 같다.



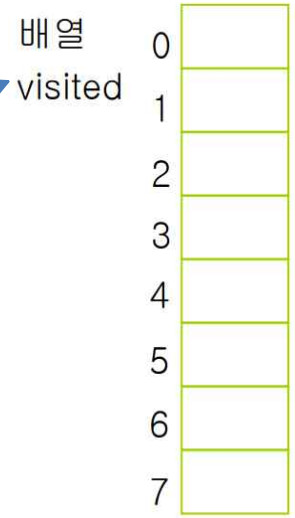
깊이우선 탐색

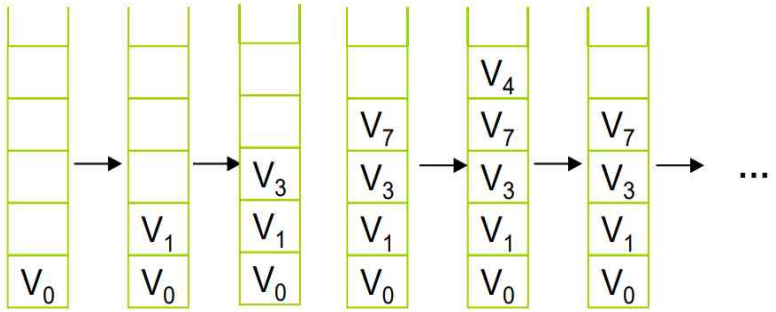


깊이우선 탐색 (Depth First Search : DFS)

DFS 알고리즘

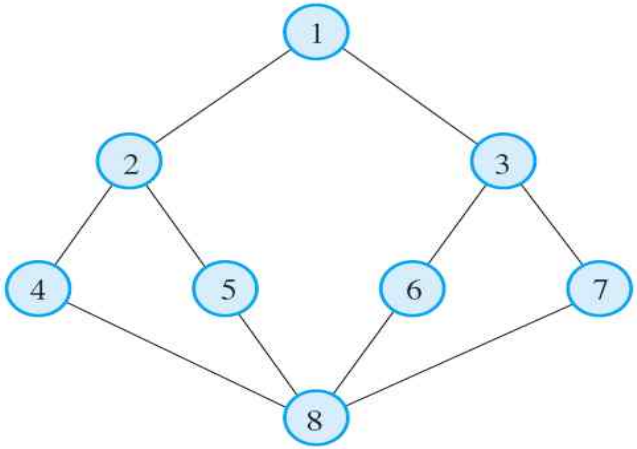
```
void dfs(int v)
   /* G = (V, E)가 n개의 정점을 가진 그래프이고 처음에는 False값으로 행렬
   visited[n]이 주어졌다고 할 때, 이 알고리즘은 정점 v로부터 도달 가능한 모든 정
   점들을 방문한다. */
{
   int w;
   visited[v] = TRUE;  /* 방문지 표시 */
   for(each vertex w adjacent to v)  /* 인접행렬 또는 입접리스트 탐색 */
      if(!visited[w]) dfs(w);
} /* dfs */
```





Stack의 변화

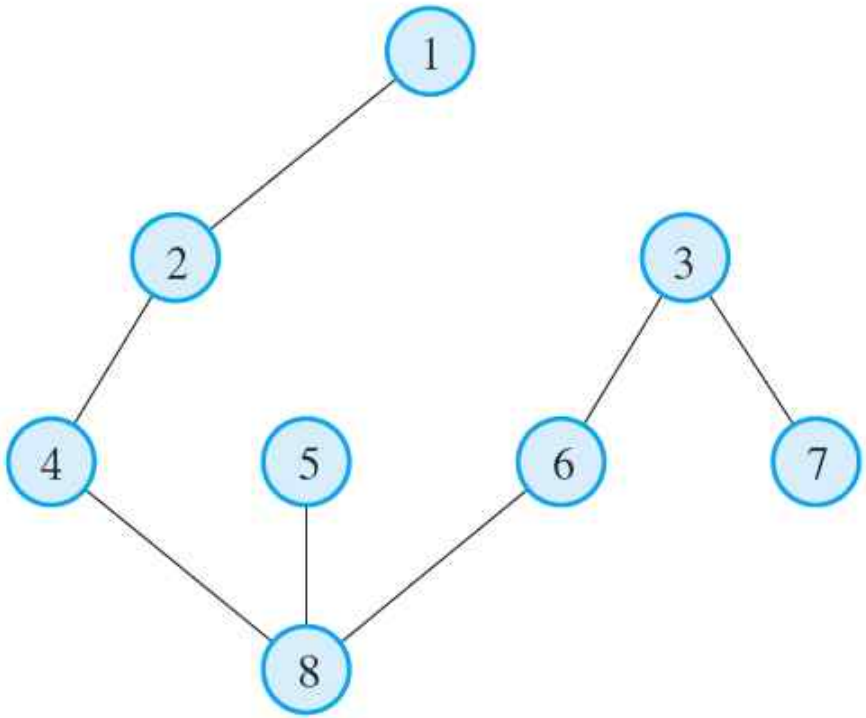
깊이우선 탐색



(1) 현재 위치는 정점 1이고, 정점 2, 3을 방문하지 않았으므로 우선은 정점 2로 이동한다.
(2) 현재 위치는 정점 2이고, 정점 1을 방문했으며, 정점 4, 5를 방문하지 않았으므로 우선은 정점 4로 이동한다.
(3) 현재 위치는 정점 4이고, 정점 2를 방문했으며, 정점 8을 방문하지 않았으므로 정점 8로 이동한다.
(4) 현재 위치는 정점 8이고, 정점 4를 방문했으며, 정점 5, 6, 7을 방문하지 않았으므로 우선은 정점 5로 이동한다.
정점 5로 이동한 후에 정점 2와 정점 8은 이미 방문된 곳이므로 더이상 갈곳이 없으니 전에 방문한 정점으로 돌아간다.
(5) 현재 위치는 정점 8이고, 정점 4, 5를 방문했으며, 정점 6, 7을 방문하지 않았으므로 우선은 정점 6로 이동한다.
(6) 현재 위치는 정점 6이고, 정점 8을 방문했으며, 정점 3을 방문하지 않았으므로 정점 3으로 이동한다.
(7) 현재 위치는 정점 3이고, 정점 1, 6을 방문했으며, 정점 7을 방문하지 않았으므로 정점 7로 이동한다.
(8) 현재 위치는 정점 7이고, 정점 3을 방문했으며, 정점 8번 노드는 이미 방문된 곳이므로 더이상 진행하지 않는다.

깊이우선 탐색

DFS의 결과는 탐색은 1,2, 4, 8, 5, 6, 3, 7의 순으로 이루어짐



깊이우선 탐색

•장점
- 단지 현 경로상의 노드들 만을 기억하면 되므로 저장공간의 수요가 비교적 적다.
- 목표 노드가 깊은 단계에 있을 경우 해를 빨리 구할 수 있다.

•단점
- 해가 없는 경로에 깊이 빠질 가능성이 있다. 따라서 실제의 경우 미리 지정한 임의의 깊이 까지만 탐색하고 목표 노드를 발견하지 못하면 다음의 경로를 따라 탐색하는 방법이 유용할 수 있다. ➔ 깊이 제한 !

너비우선 탐색

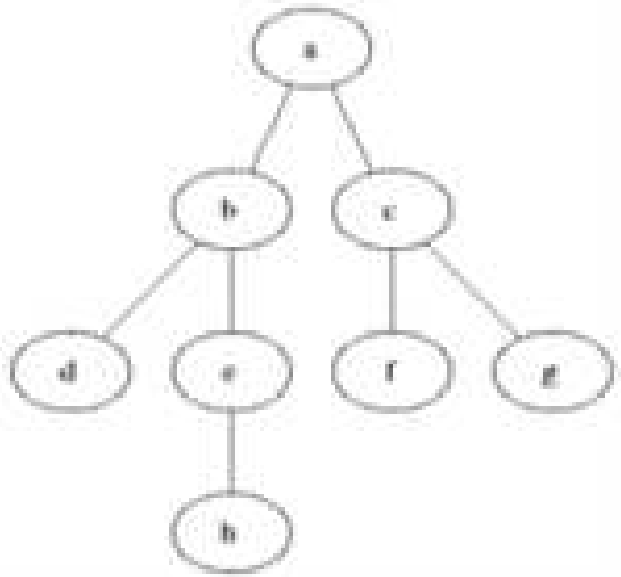
친척들에게 안부전화를 드리는 순서를, BFS 로 고려하면? 1촌 2촌→3촌→



그림 출처: 부교재 이산수학 express,김대수 저

너비우선 탐색

탐색방법 : 시작 정점을 방문한 후, 시작 정점에 인접한 모든 정점들을 우선 방문하는 방법이다. 더 이상 방문하지 않은 정점이 없을 때까지 방문하지 않은 모든 정점들에 대해서도 너비 우선 검색을 적용한다



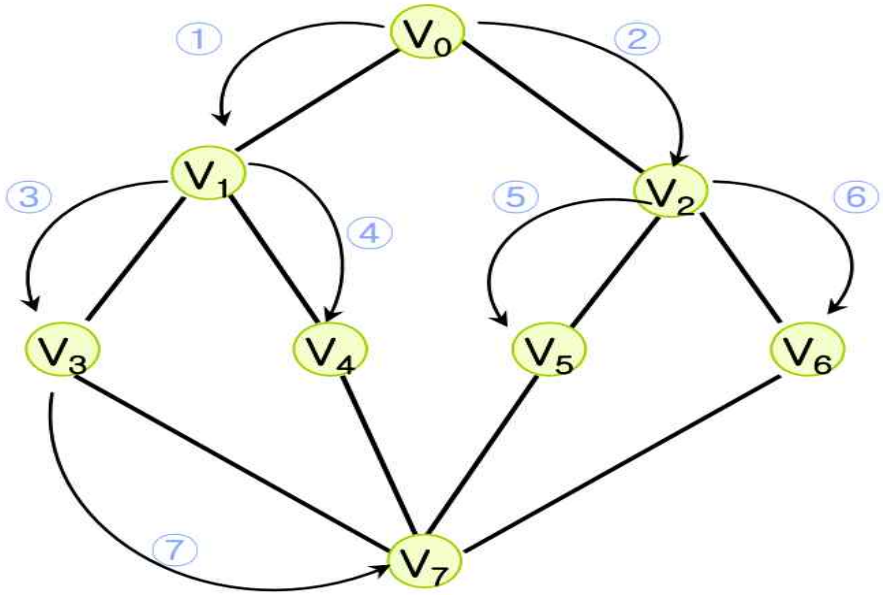
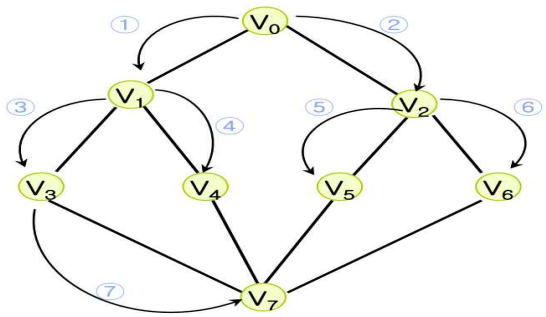


그림 출처: wiki백과

너비우선 탐색



너비우선 탐색 (Breadth First Search : BFS)

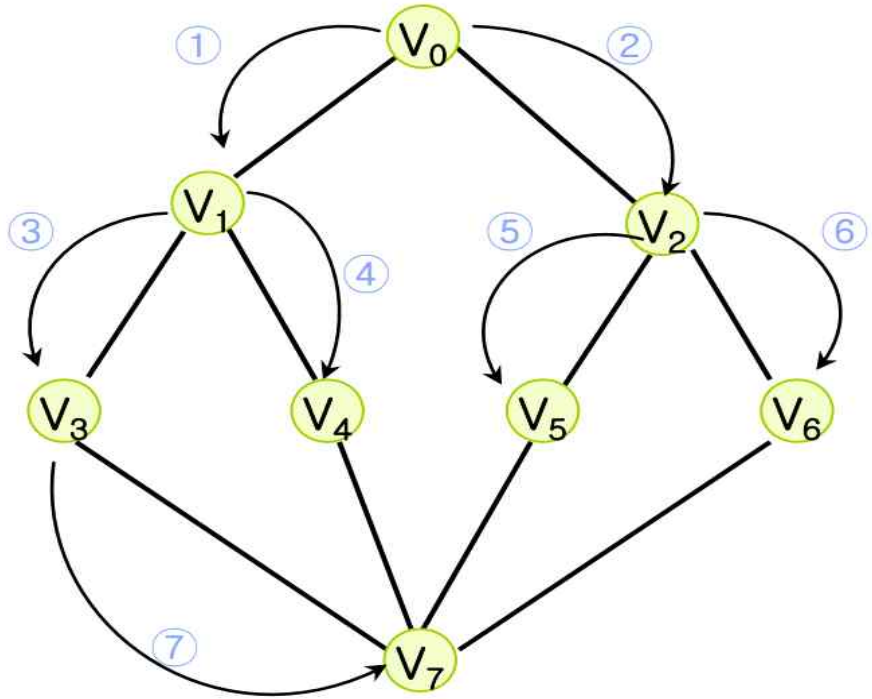
방법 : 자동차로 여행할 경우 가까운 곳부터 방문하는 방법이다. 즉 어느 지점에서 가까운 곳을 방문하고 다음 방문 가능 지점은 모두 저장해둔다. 다음 방문지는 항상 저장해 둔 곳에서 순서대로 꺼낸다. 예를 들어 종로 1가에서 시작하여 종로의 버스정류장을 모두 방문해야 한다고 하면 종로 1가의 한 정류장 거리를 다 방문하고 끝나면 두 정류장 거리에 있는 정류장을 모두 방문하고 이러한 순서로 진행한다. 두 정류장 거리에 있는 정류장들은 첫번째 정류장을 방문할 경우 기억을 해둔다. 먼저 기억된 정류장이 다음 번에 먼저 방문을 하게 된다. 즉 큐 자료구조를 이용하여 다음 방문지를 선택한다.

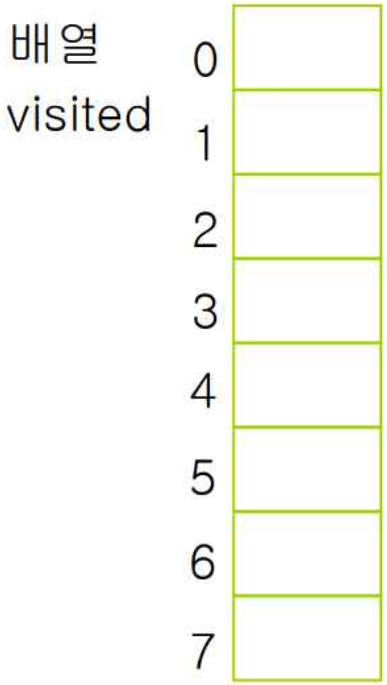
- 너비 우선 탐색은 처음에 방문한 정점과 인접한 정점들을 차례로 방문한다는 점에서 깊이 우선 탐색과 차이가 있음

1. 먼저 주어진 정점 v를 출발점으로 하여 이를 방문한다.
2. 그리고 v에 인접한 정점 w들을 먼저 모두 방문하고 그 다음으로 w에 인접하고 아직 방문하지 않은 정점들을 모두 방문한다.
3. 위의 과정을 반복하여 더 이상 방문할 노드가 없을 때까지 계속하여 방문한다.

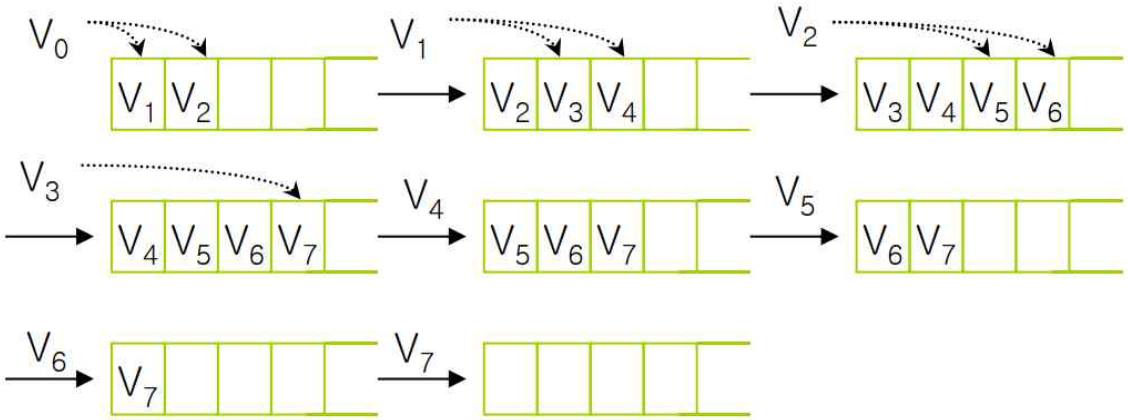
➢ 구현 : 큐(queue)를 사용

너비우선 탐색





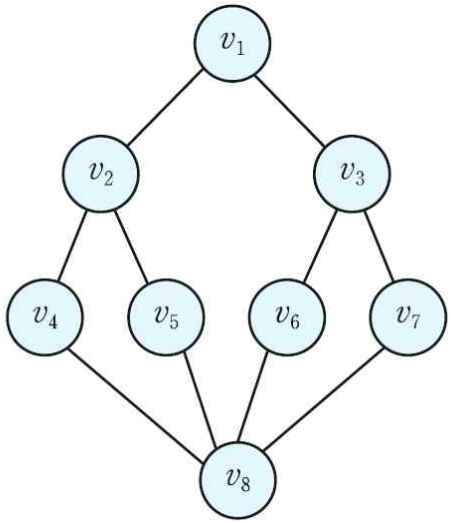
큐의 형태



현재 위치에서 갈 수 있는 정점을 모두 큐에 넣는 방식으로
큐에 넣음과 동시에 방문했다고 체크해야함.

너비우선 탐색

다음의 너비우선 탐색을 해보자.

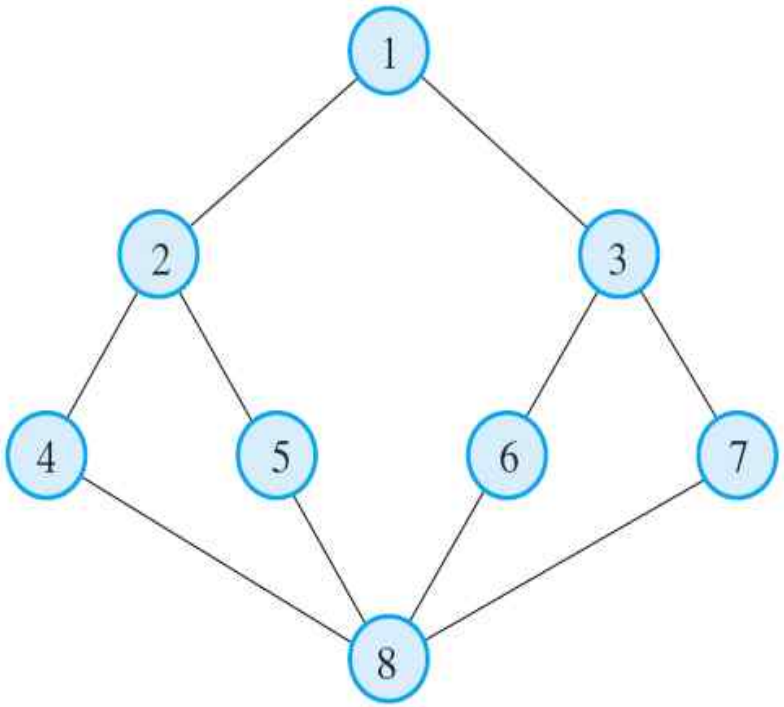


[그림 7-9] 그래프

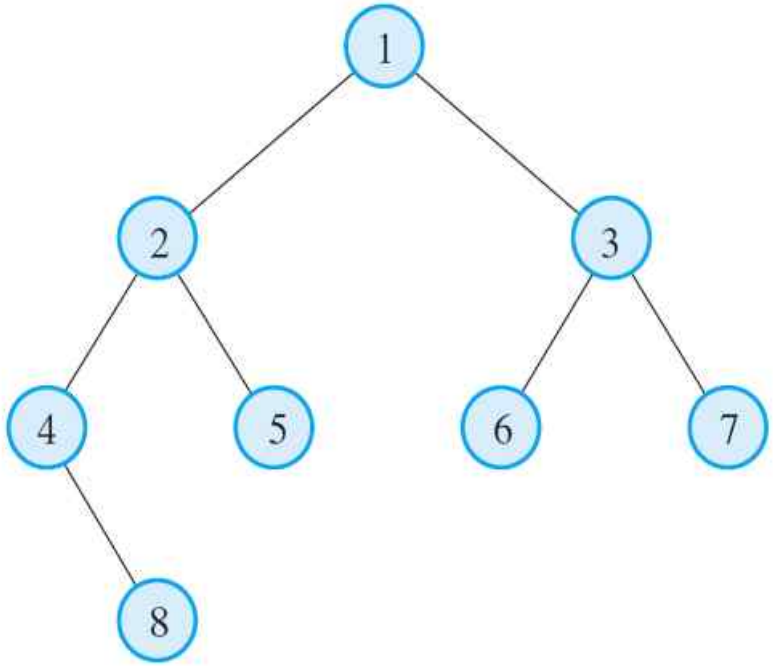
- v_1을 출발점으로 하여 너비우선 탐색에 의해서 방문하면 $v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8$이 된다.

너비우선 탐색

다음의 너비우선 탐색을 해보자.



주어진 그래프

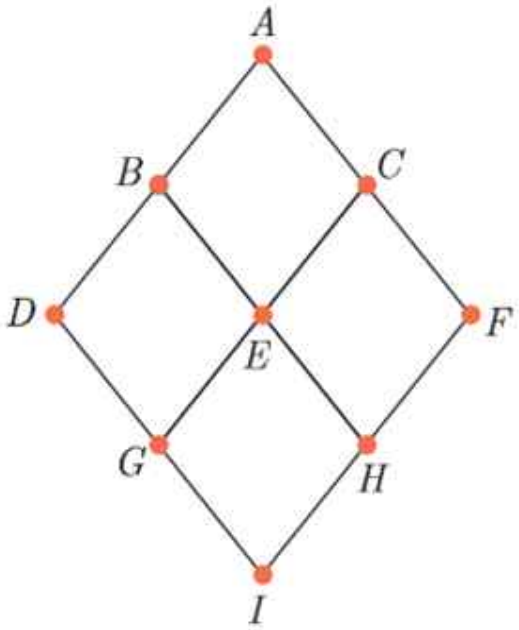


BFS 수행 결과

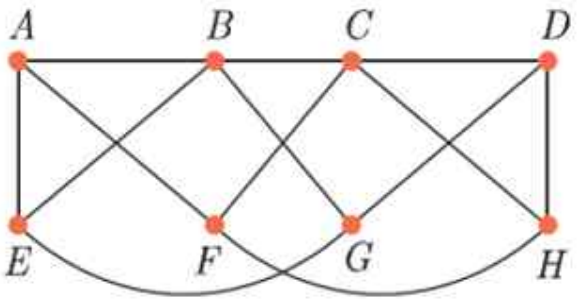
너비우선 탐색

다음의 사전적 순서로 너비우선 탐색하고 신장부분그래프를 그려라.

(a)

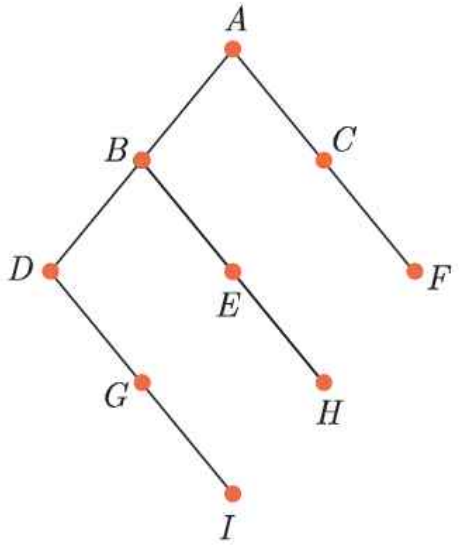


(b)

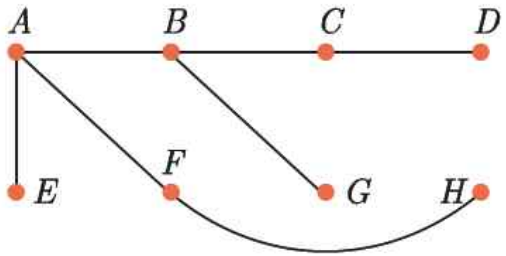


너비우선 탐색

(a) $A-B-C-D-E-F-G-H-I$이고, 탐색 결과인 신장부분그래프는 다음과 같다.



(b) $A-B-E-F-C-G-H-D$이고, 탐색 결과인 신장부분그래프는 다음과 같다.

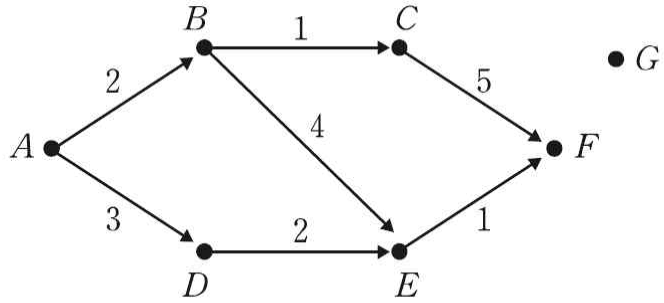


너비우선 탐색

- 장점
  - 출발 노드에서 목표 노드까지의 최단 길이 경로를 보장한다.

- 단점
  - 경로가 매우 길 경우에는 탐색 가지가 급격히 증가함에 따라 보다 많은 기억 공간을 필요로 하게 된다.
  - 해가 존재하지 않는다면 유한 그래프(finite graph)의 경우에는 모든 그래프를 탐색한 후에 실패로 끝난다.

최단 경로



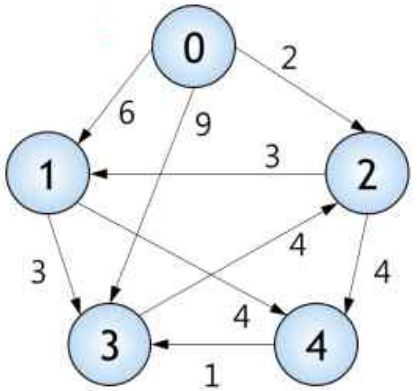
[그림 7-10] 가중치 그래프

최단 경로

- 가중치 그래프를 이용한 두 정점 사이에 최단 경로(shortest path)를 찾는 문제를 생각해보자.
  - 두 정점 사이에 최단 경로의 길이는 가중치들의 합 중에서 가장 작은 값을 가지는 경로를 말한다.
  - 정점 u에서 정점 v 사이에 최단 경로를 두 정점 사이의 거리라고 한다.
  - 만일 어떤 두 정점 사이에 최단 경로가 없다면 이때의 거리는 무한대(∞)라고 가정한다.
- 그러면 최단 경로를 어떻게 하면 찾을 수 있을까?
  - 그것에 대한 명백한 해답은 모든 경로에 대해서 거리를 계산하는 것이다.
  - 그러나 이 방법은 n 개의 정점들이 있을 때, 정점 a로부터 정점 b로의 경로는 약 $(n-1)!$ 이상이 존재하므로, 만일 $n = 30$일 경우에 경로를 컴퓨터로 계산한다고 하더라도 엄청난 시간을 요구할 것이다.
  - 그러므로 모든 경우의 경로를 구해서 계산한다는 것은 무리이다.

최단 경로

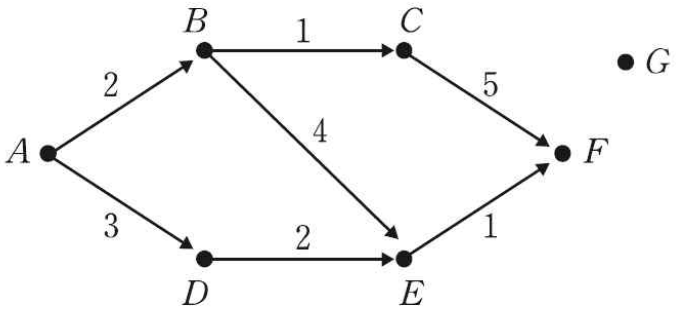
- 이를 해결하는 알고리즘의 하나가 다익스트라(Dijkstra)의 알고리즘.
  - 다익스트라는 네덜란드 사람으로 이론 물리학자이며 1972년 ACM으로부터 그 유명한 튜링(Alan Turing) 상을 수상.
    - 튜링 상은 컴퓨터 분야에서의 노벨상
  - 출발점으로부터 모든 정점들의 최단거리를 계산할 수 있다.
  - (아이디어) **출발점에서 가까운 도시(정점)부터 차례대로 최단 경로를 찾아 나감**. 출발점으로 부터 가장 가깝다고 한번 결정된 최단 경로는 절대 바뀌지 않으며 출발점에서 더 먼 도시의 최단경로를 구할 때 이용



최단경로	길이
0 → 2	2
0 → 2 → 1	5
0 → 2 → 4	6
0 → 2 → 4 → 3	7

최단 경로

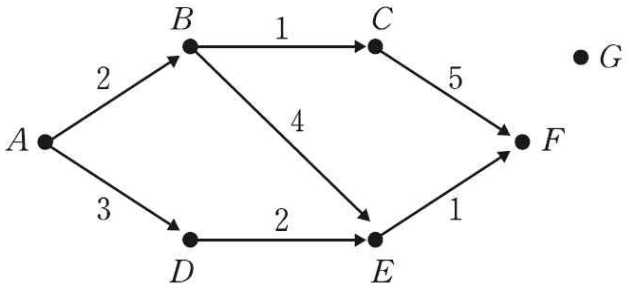
- (예) 정점 A에서 F로 가는 최단 경로는 ?
  - 직관적으로, [그림 7-10]에서 A부터 F까지의 최단 경로의 가중치는 6이며 정점 A, D, E, F로 구성됨을 알 수 있다.



[그림 7-10] 가중치 그래프

  - 정점 A로부터 다른 정점까지 최단 경로를 찾는 데 필요한 단계들
    - 가중치 그래프에서 각각의 정점 X에 대해서 $d[X]$ (or D[x]) 는 출발 정점 A로부터 X까지의 거리를 나타낸다고 하자. (ex: $d[3]=3$)
    - 첫 번째로 초기값을 각 $d[X]$에 할당하고 정점을 한 번 이상 탐색하게 되면 그때 $d[X]$의 값을 바꾼다.
    - 한 정점을 이미 탐색했다면 그 정점은 앞으로 찾아가는 정점에서 제외된다.

최단 경로

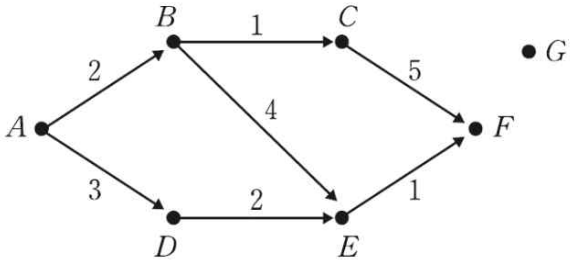


[그림 7-10] 가중치 그래프

[표 7-1] A로부터 최단 경로 찾기

단계	찾아갈 정점	정점에 대한 거리							탐색하지 않은 정점
		A	B	C	D	E	F	G	
0	A	0	2	∞	3	∞	∞	∞	B, C, D, E, F, G
1	B	0	2	3	3	6	∞	∞	C, D, E, F, G
2	D	0	2	3	3	5	∞	∞	C, E, F, G
3	C	0	2	3	3	5	8	∞	E, F, G
4	E	0	2	3	3	5	6	∞	F, G
5	F	0	2	3	3	5	6	∞	G

최단 경로



[그림 7-10] 가중치 그래프

[표 7-1] A로부터 최단 경로 찾기

단계	찾아갈 정점	정점에 대한 거리							탐색하지 않은 정점
		A	B	C	D	E	F	G	
0	A	0	2	∞	3	∞	∞	∞	B, C, D, E, F, G
1	B	0	2	3	3	6	∞	∞	C, D, E, F, G
2	D	0	2	3	3	5	∞	∞	C, E, F, G
3	C	0	2	3	3	5	8	∞	E, F, G
4	E	0	2	3	3	5	6	∞	F, G
5	F	0	2	3	3	5	6	∞	G

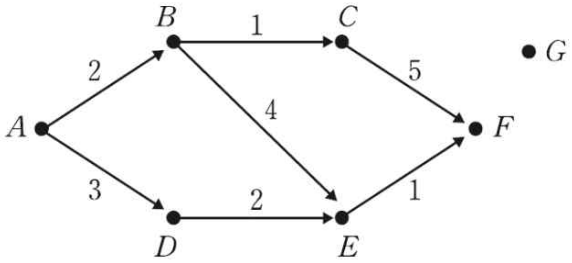
❖ 단계 0

- $X = A$일 때 $d[X]$의 값은 0, (A, X)에 간선이 없으면 ∞, (A, X)에 간선이 있으면 그 간선에 있는 가중치를 부여한다.
- 정점 A가 가장 가까운 정점이므로 이 정점을 제일 먼저 탐색하고 다른 정점들은 탐색하지 않는다.

❖ 단계 1

- 탐색하지 않은 정점 중에서 **B가 A에 가장 가까운** 이므로 다음 정점으로 탐색하고, 탐색하지 않은 정점에서 제거한다.
- 탐색하지 않은 모든 Y에 대해서 B를 통과해서 Y까지의 새로운 거리가 이전에 계산된 거리보다 짧은가를 검사한다.
- C와 E에 도달할 수 있으므로 거리를 다시 계산해서 3과 6으로 쓴다.

최단 경로



[그림 7-10] 가중치 그래프

[표 7-1] A로부터 최단 경로 찾기

단계	찾아갈 정점	정점에 대한 거리							탐색하지 않은 정점
		A	B	C	D	E	F	G	
0	A	0	2	∞	3	∞	∞	∞	B, C, D, E, F, G
1	B	0	2	3	3	6	∞	∞	C, D, E, F, G
2	D	0	2	3	3	5	∞	∞	C, E, F, G
3	C	0	2	3	3	5	8	∞	E, F, G
4	E	0	2	3	3	5	6	∞	F, G
5	F	0	2	3	3	5	6	∞	G

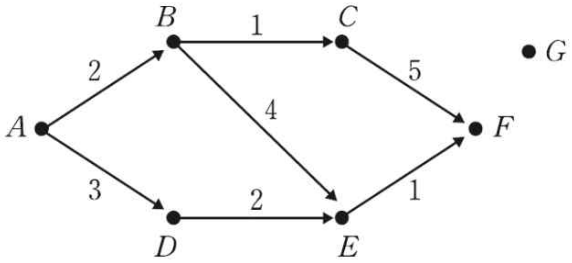
❖ 단계 2

- 출발 A로 부터 나머지 탐색하지 않은 정점들 중에서 C와 D가 거리가 3으로 가장 가깝다.
- 임의로 D를 선택해서 탐색한다. E는 D를 통한 경로가 되므로 거리에 대한 값이 6에서 5로 바뀐다.

❖ 단계 3

- C는 A에 가장 가까운 정점이므로 C를 탐색한다.
- F가 C를 통해서 도달되므로 거리를 8로 바꾼다.

최단 경로



[그림 7-10] 가중치 그래프

[표 7-1] A로부터 최단 경로 찾기

단계	찾아갈 정점	정점에 대한 거리							탐색하지 않은 정점
		A	B	C	D	E	F	G	
0	A	0	2	∞	3	∞	∞	∞	B, C, D, E, F, G
1	B	0	2	3	3	6	∞	∞	C, D, E, F, G
2	D	0	2	3	3	5	∞	∞	C, E, F, G
3	C	0	2	3	3	5	8	∞	E, F, G
4	E	0	2	3	3	5	6	∞	F, G
5	F	0	2	3	3	5	6	∞	G

❖ 단계 4

- 출발 A로 부터 나머지 탐색하지 않은 정점들 E,F,G 중에서 가장 가까운 점 E를 탐색하고 F로의 거리를 6으로 바꾼다.

❖ 단계 5

- F를 탐색한다.
- 도달될 수 있는 정점이 없으므로 G는 탐색되지 않는다.

최단 경로

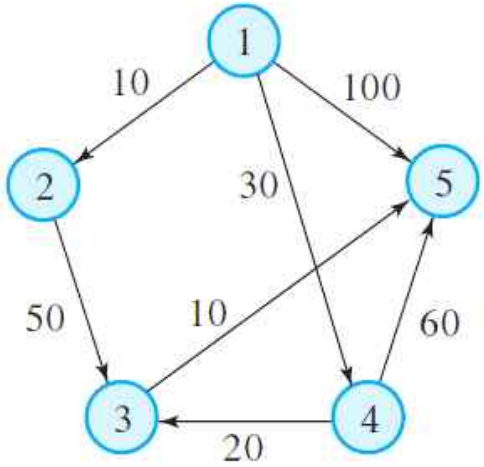
다익스트라 알고리즘

```
void Dijkstra()
    /* 이 알고리즘은 정점 1에서 방향 그래프의 모든 정점으로의 최단 거리를 계산한다.*/
{
    S = {1};     // 출발점 1, S= 출발점으로 부터 최단거리를 찾은 정점들 집합
    for( i = 2 ; i <= n ; i++ ) // 출발점 1로 부터 각 정점까지의 거리를 초기화
        D[i] = C[1, i];  /* D값을 초기화한다 */
    for( i = 1 ; i <= n-1 ; i++ )
    {           // 출발점 1로 부터 S 집합 밖의 점들 중에서 가장 가까운 점 w 찾기
        choose a vertex w in V - S such that
            D[w] is a minimum;
        add w to S;  // 출발점 1로 부터 최단거리를 찾은 점 w를 S 집합에 추가
        for(each vertex v in V - S)
            D[v] = min (D[v], D[w] + C[w, v]);
    }                // 최단거리를 찾은 점 w를 이용하여
                       출발점 1로 부터 S 밖의 점들까지의 거리 갱신
} /* Dijkstra */
```

최단 경로

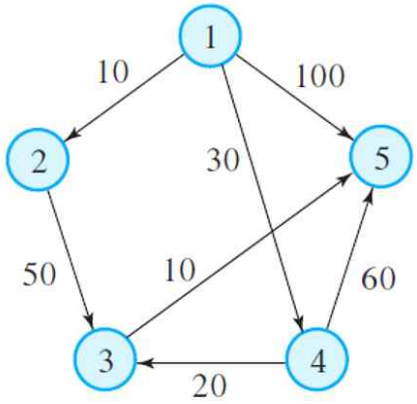
예제)

다익스트라 알고리즘을 다음과 같은 가중 그래프에 적용해보자.



풀이 처음 단계에서 $S = \{1\}$, $D[2] = 10$, $D[3] = \infty$, $D[4] = 30$, $D[5] = 100$이 된다. for 루프의 첫 번째 반복에서 $w = 2$가 최소의 D값을 가진 점으로 선택된다. 그러면 $D[3] = \min(\infty, 10 + 50) = 60$이 되고 $D[4]$와 $D[5]$는 변동이 없다. 왜냐하면 점 1에서 직접 가는 값이 점 2를 경유하는 것보다 값이 작기 때문이다. for 루프에서의 각 반복 과정 후의 D값은 다음 표와

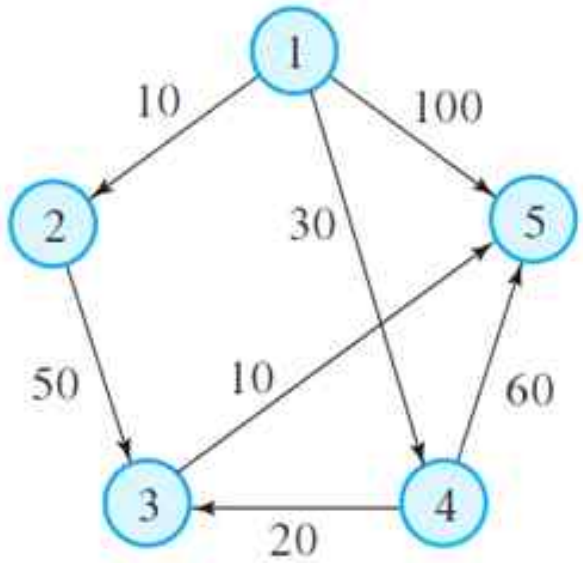
최단 경로



같으며 4번째 반복 후 점 1에서 점 2, 점 3, 점 4까지의 거리가 각각 10, 50, 30, 60으로 최종 결정된다.

단계	S	w	$D[2]$	$D[3]$	$D[4]$	$D[5]$
0	{1}	—	10	∞	30	100
1	{1, 2}	2	10	60	30	100
2	{1, 2, 4}	4	10	50	30	90
3	{1, 2, 3, 4}	3	10	50	30	60
4	{1, 2, 3, 4, 5}	5	10	50	30	60

시작정점



같으며 4번째 반복 후 점 1에서 점 2, 점 3, 점 4
30, 60으로 최종 결정된다.

알고리즘 1 다익스트라 알고리즘

```
void Dijkstra()
   /* 이 알고리즘은 정점 1에서 방향 그래프의 모든 정점으로의 최단 거리를 계산한다.*/
{
   S = {1};   V ={2,3,4,5}
   for( i = 2 ; i <= n ; i++ )
      D[i] = C[1, i];  /* D값을 초기화한다 */
   for( i = 1 ; i <= n-1 ; i++ )
   {
      choose a vertex w in V - S such that
         D[w] is a minimum;
      add w to S;
      for(each vertex v in V - S)
         D[v] = min (D[v], D[w] + C[w, v]);
   }
} /* Dijkstra */
```

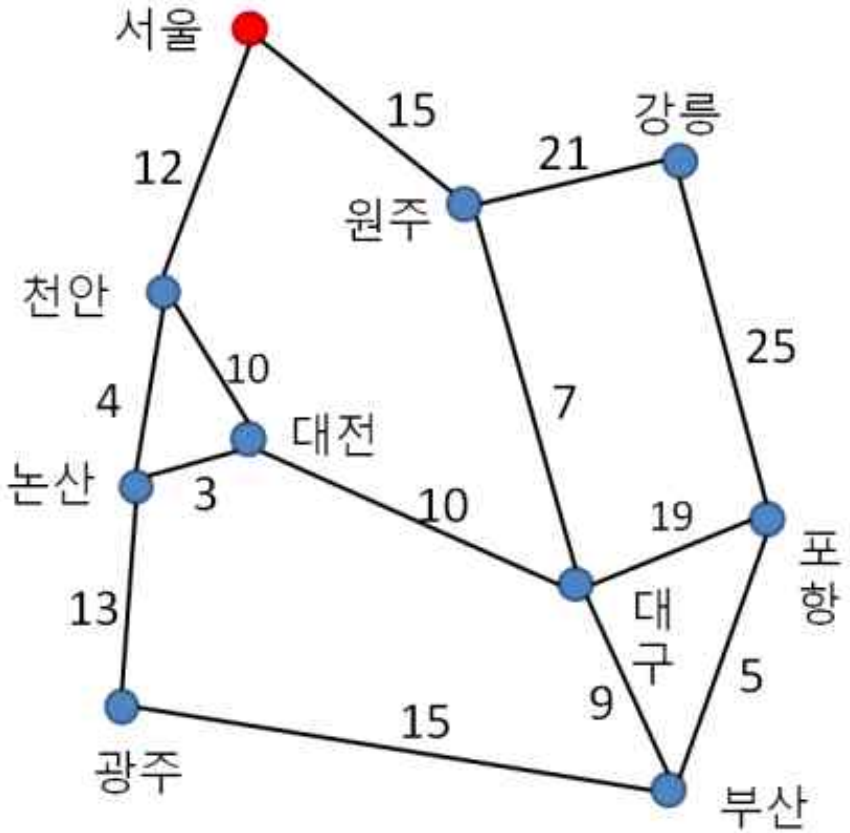
단계	S	w	$D[2]$	$D[3]$	$D[4]$	$D[5]$
0	{1}	—	10	∞	30	100
1	{1, 2}	2	10	60	30	100
2	{1, 2, 4}	4	10	50	30	90
3	{1, 2, 3, 4}	3	10	50	30	60
4	{1, 2, 3, 4, 5}	5	10	50	30	60

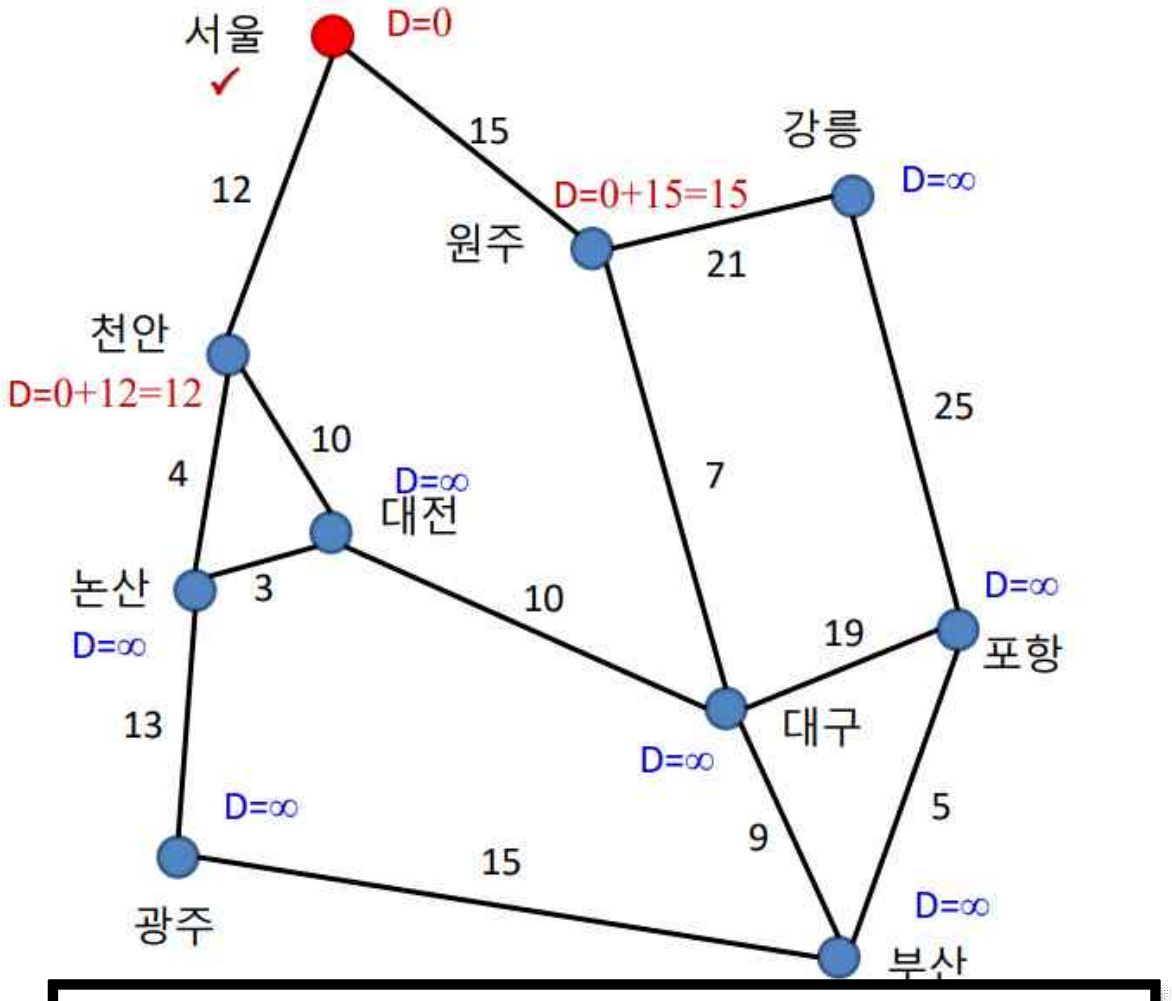
D[w]가 가장 작은 값을 갖는 w=4를 선택

4까지가는 최단거리를 알고 있을 때, 이 거리를 이용하여 다른 점들의 최단거리 갱신

최단 경로

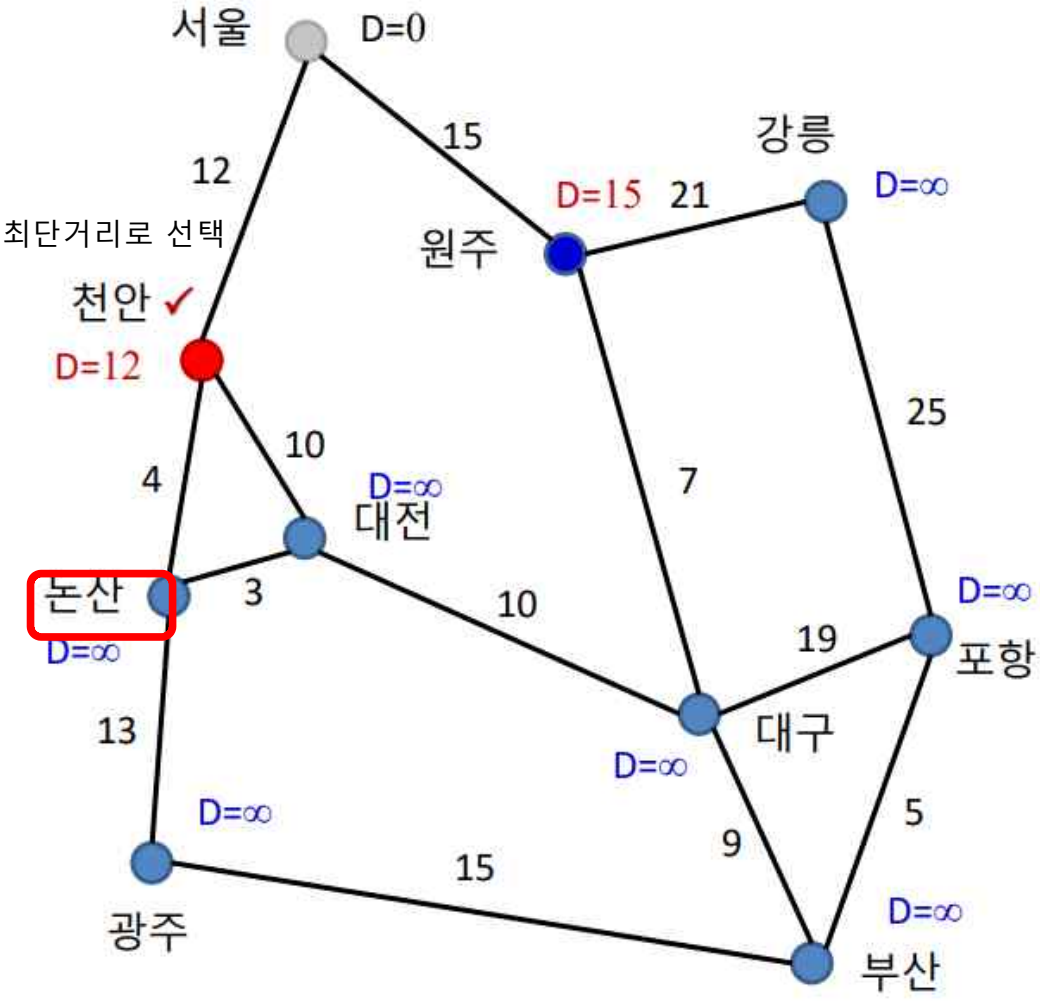
(예제) 최단 경로 찾기- 다익스트라 알고리즘 (출발점 : 서울)



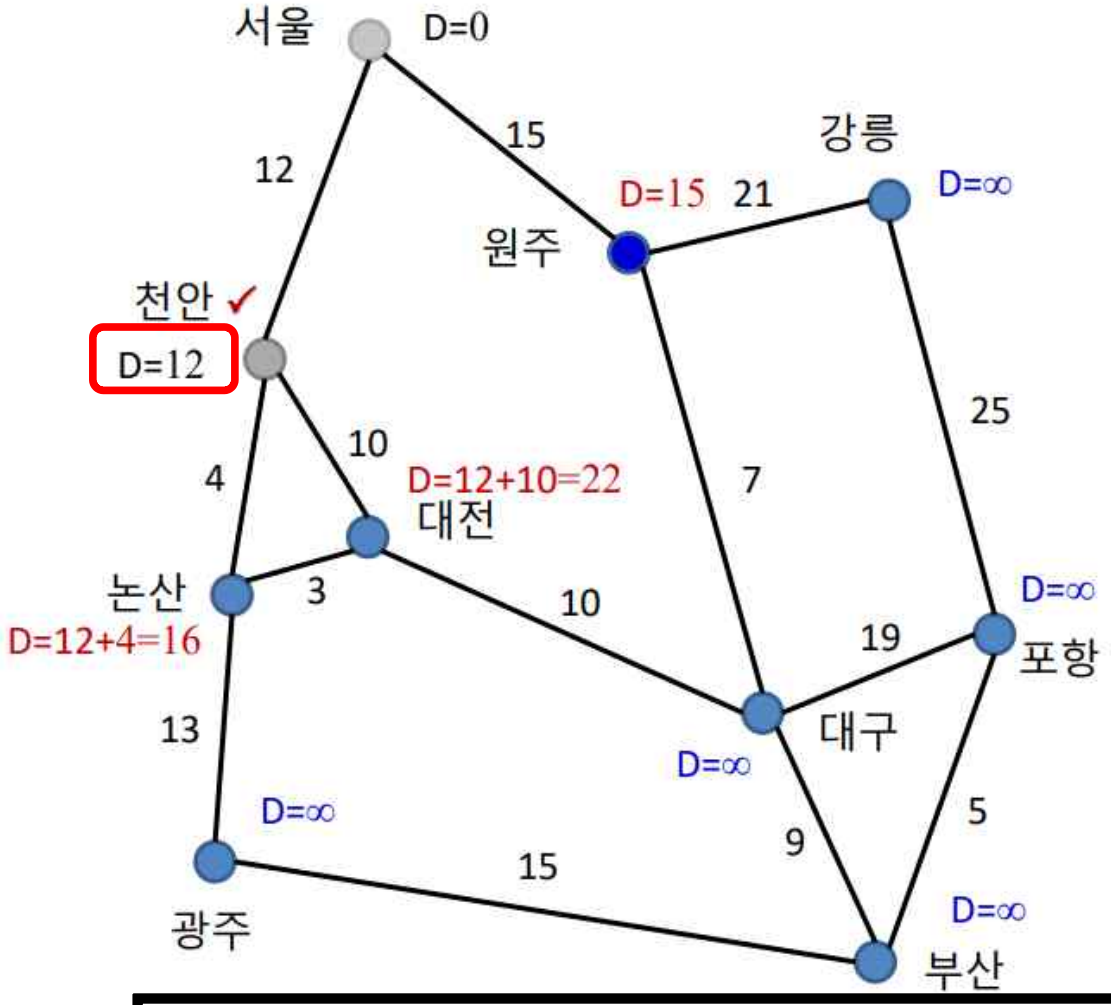


	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	∞	15	12	∞	∞	∞	∞	∞	∞]

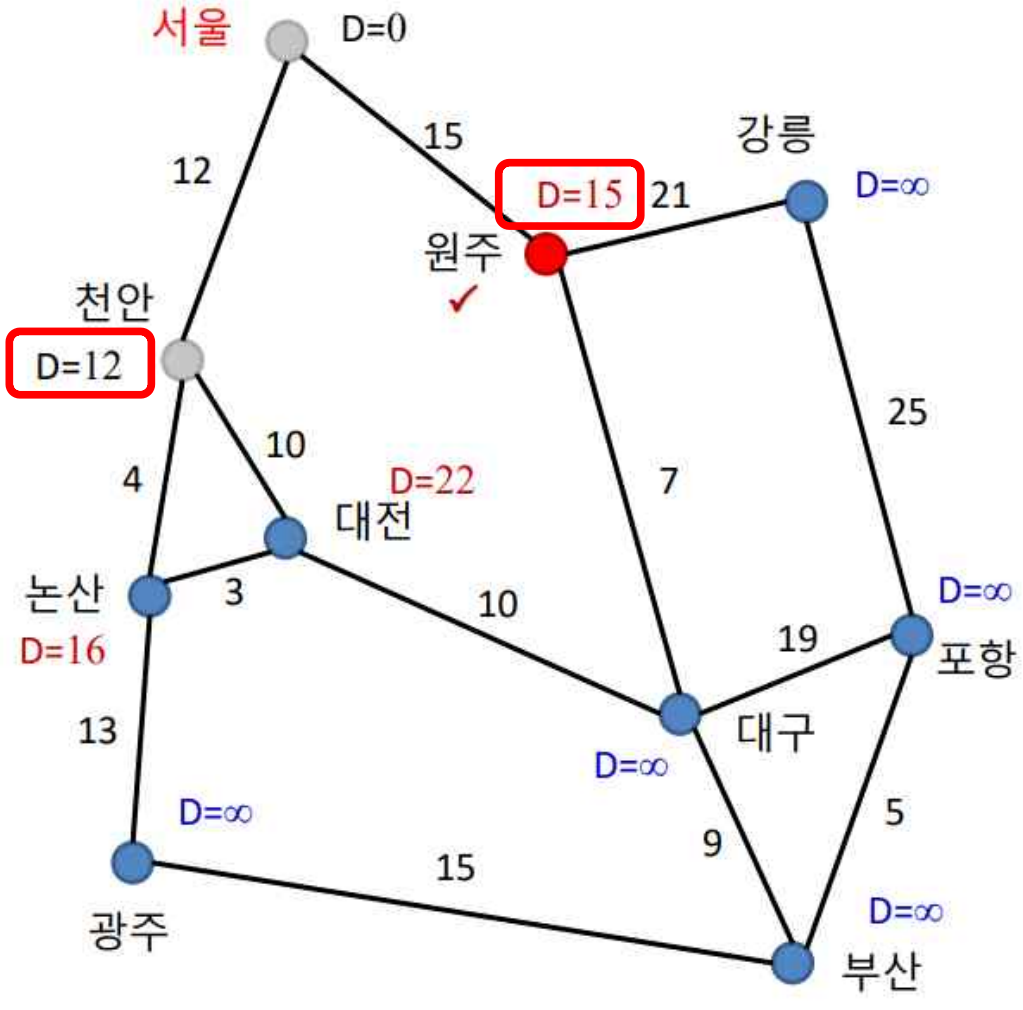
서울에서 천안으로 갈 수 있는 최단거리로 선택되었으므로 값은 바뀌지 않음



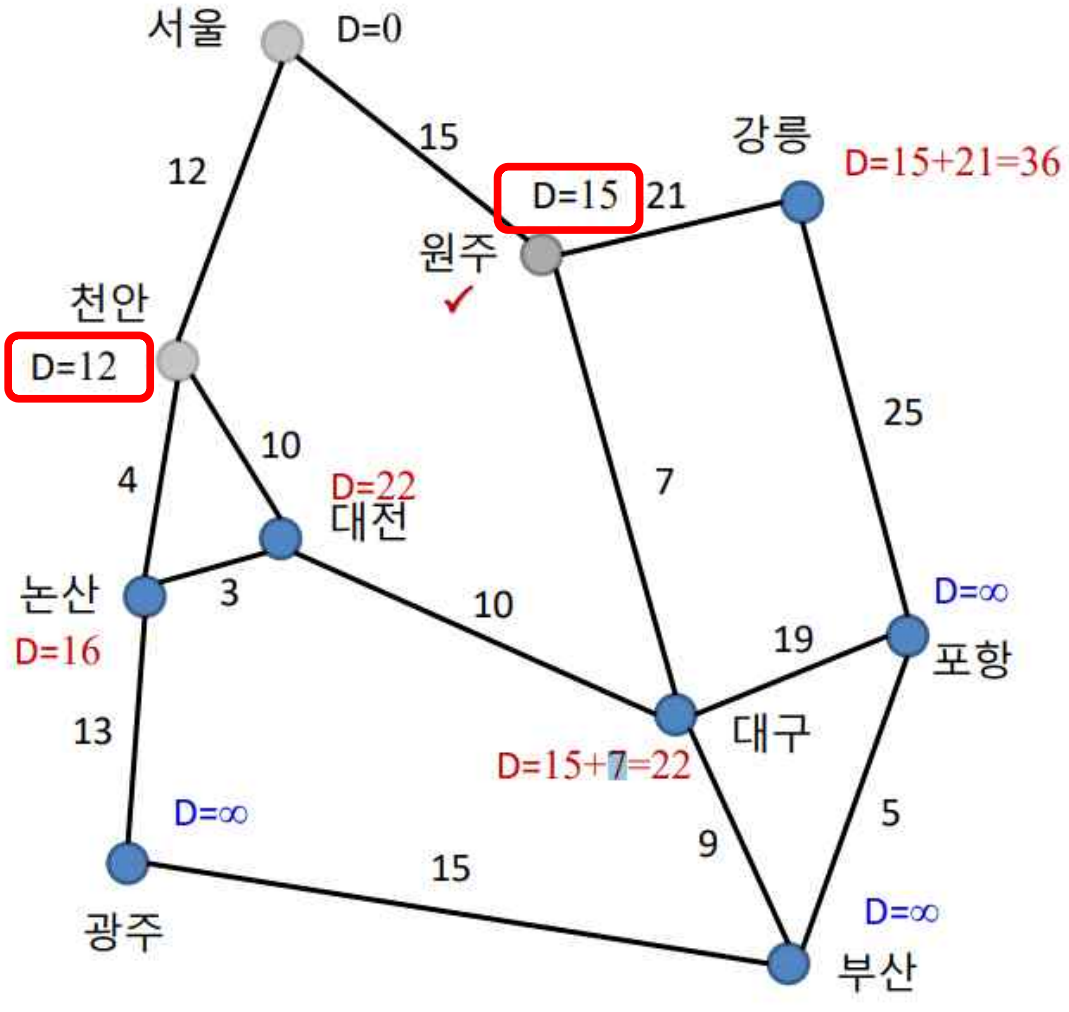
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	∞	15	12	16	22	∞	∞	∞	∞]



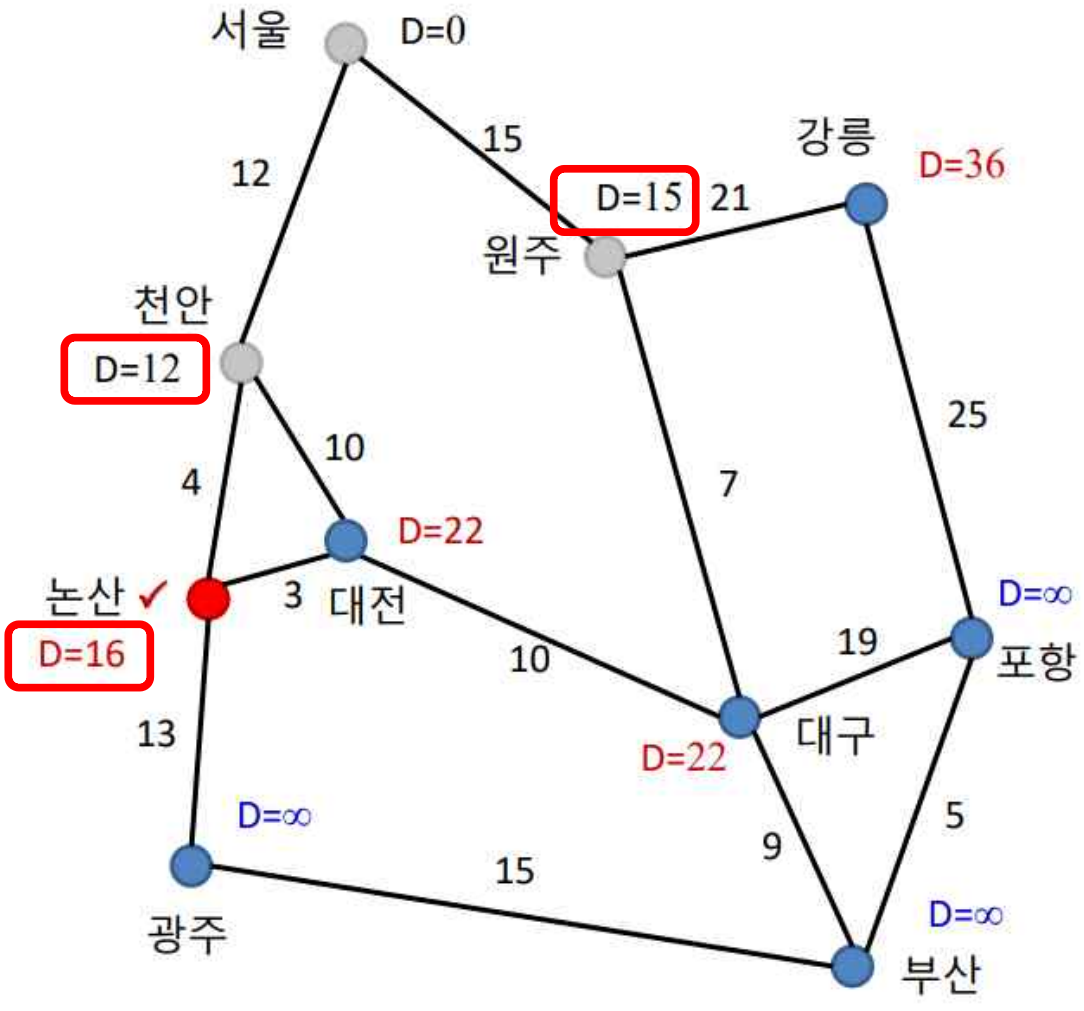
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	∞	15	12	16	22	∞	∞	∞	∞]



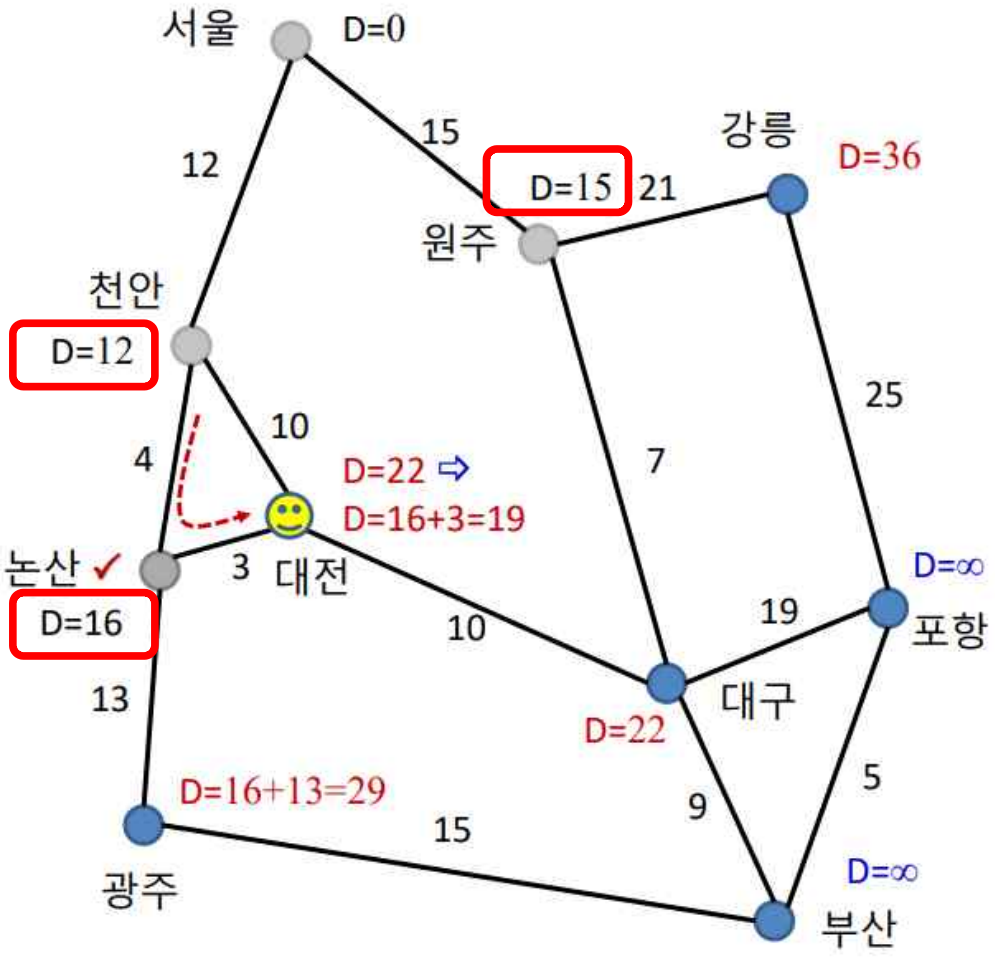
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	22	22	∞	∞	∞]



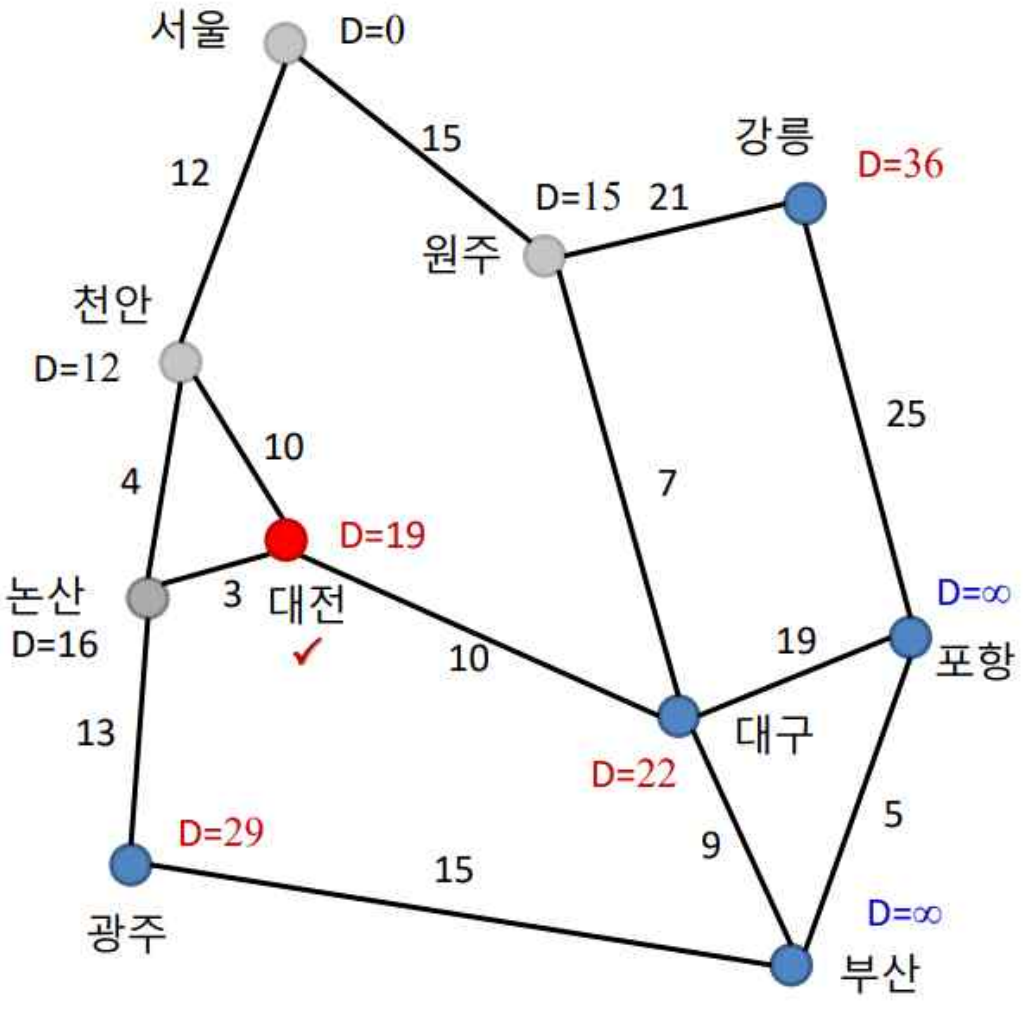
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	22	22	∞	∞	∞]

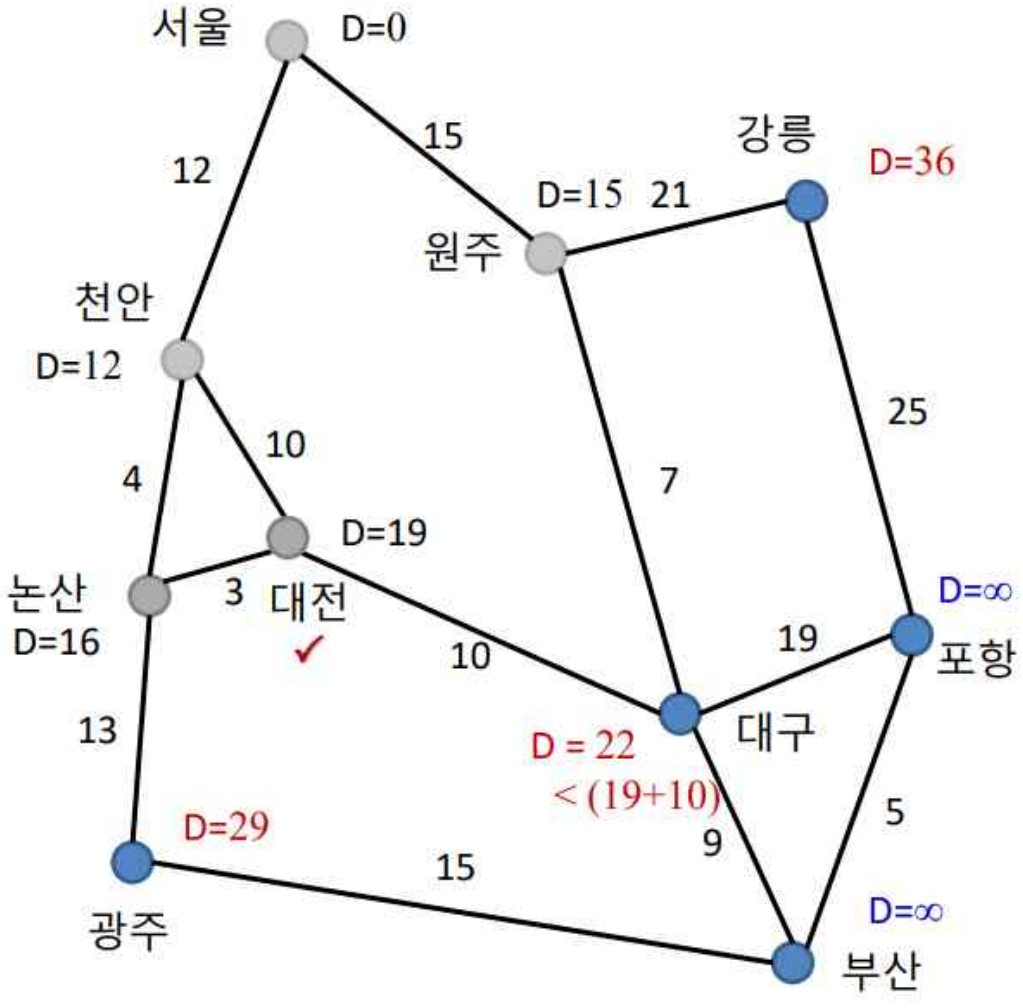


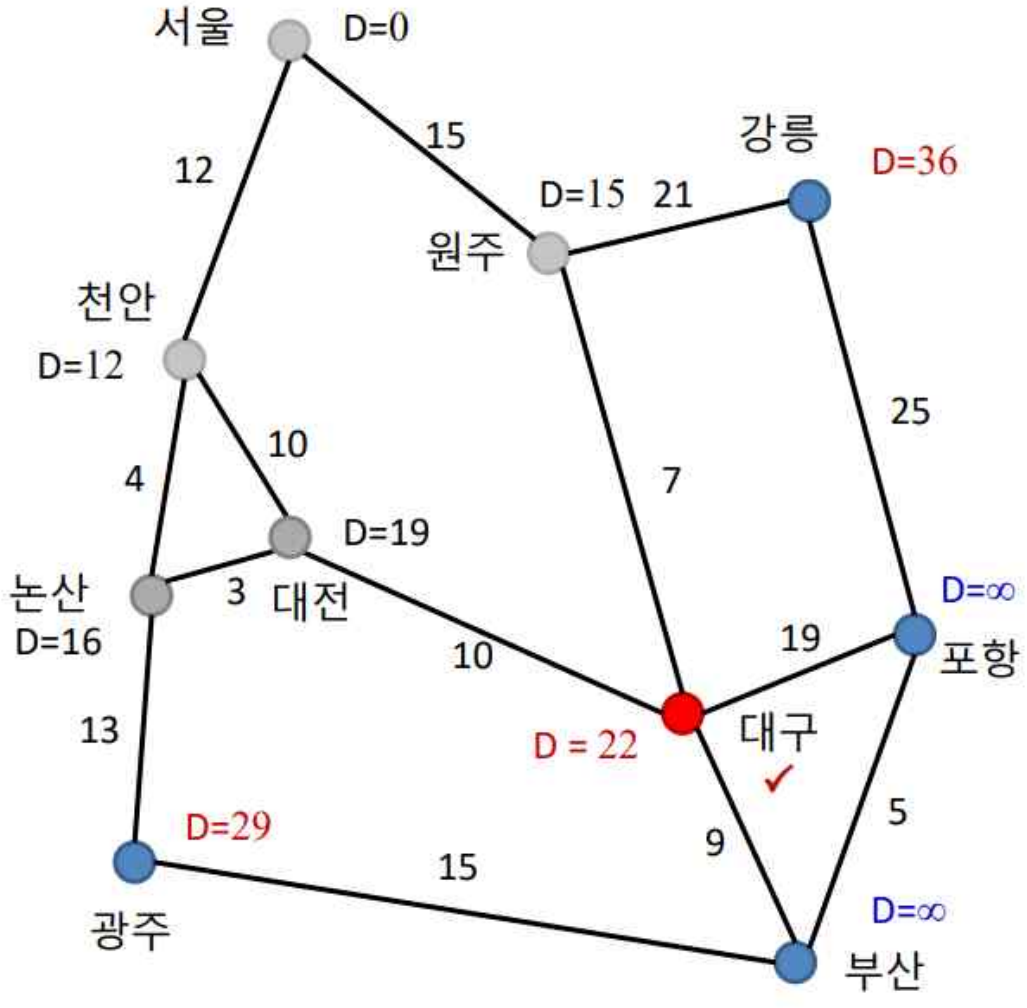
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	19	22	∞	29	∞]

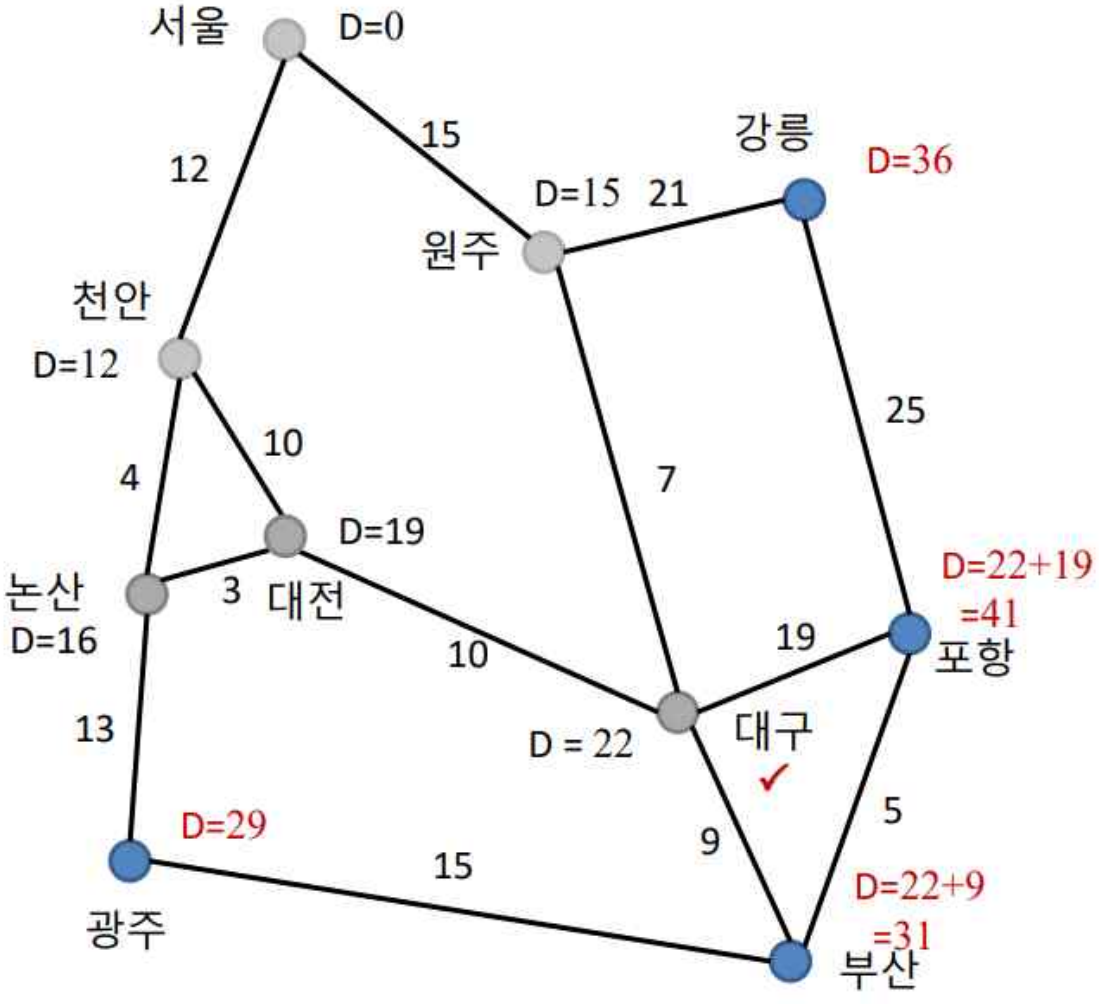


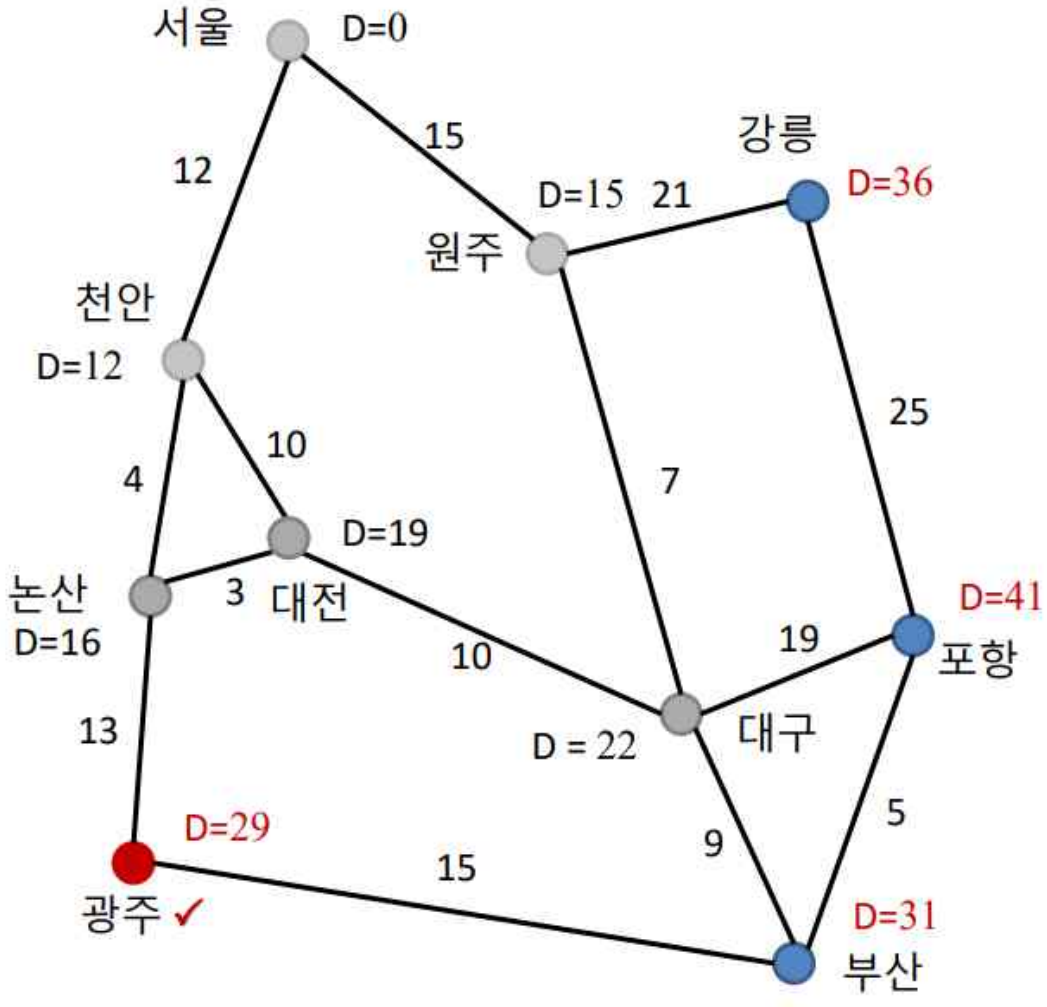
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	19	22	∞	29	∞]

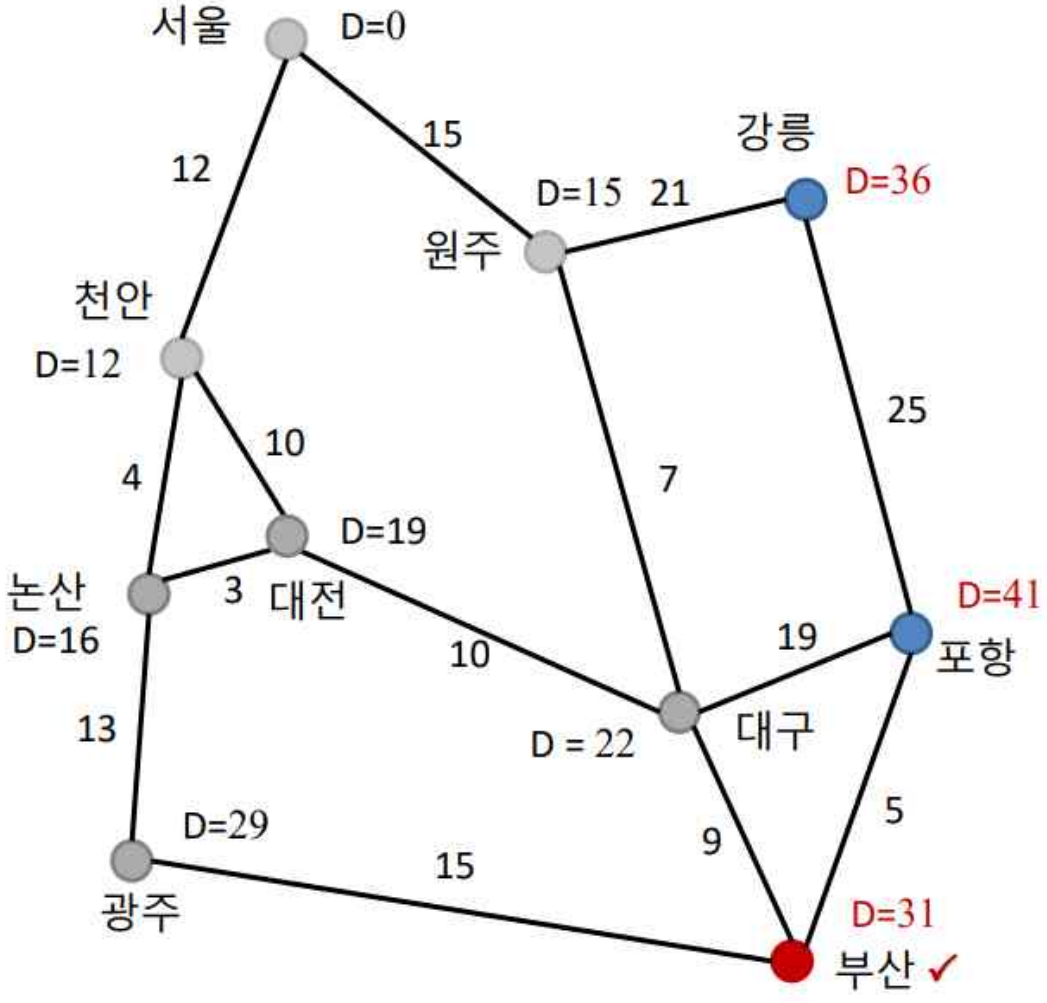

서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=∞
19
포항
10
13
대구
D=22
5
D=29
9
15
D=∞
광주
부산

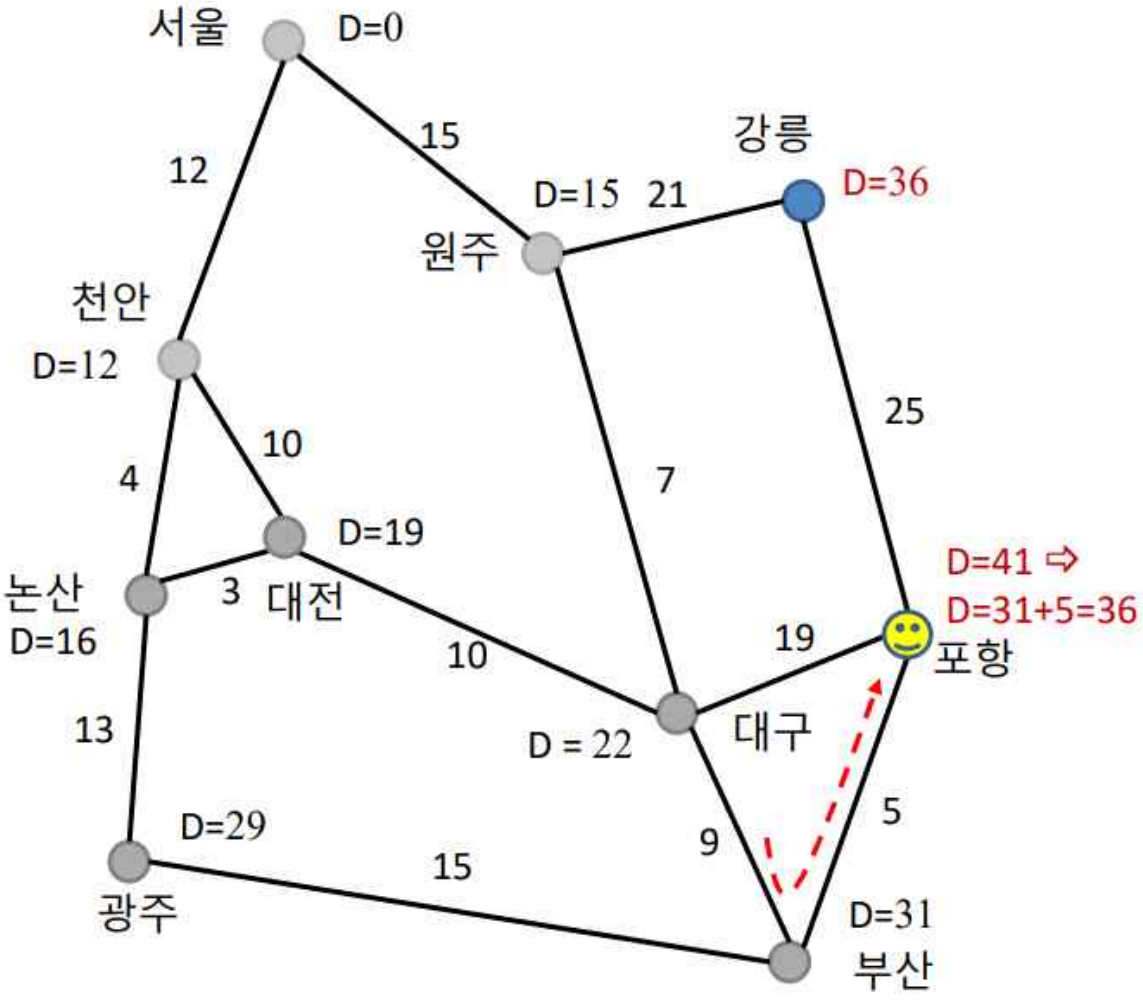

서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=∞
19
포항
10
13
대구
D = 22
< (19+10)
D=29
9
5
15
D=∞
광주
부산

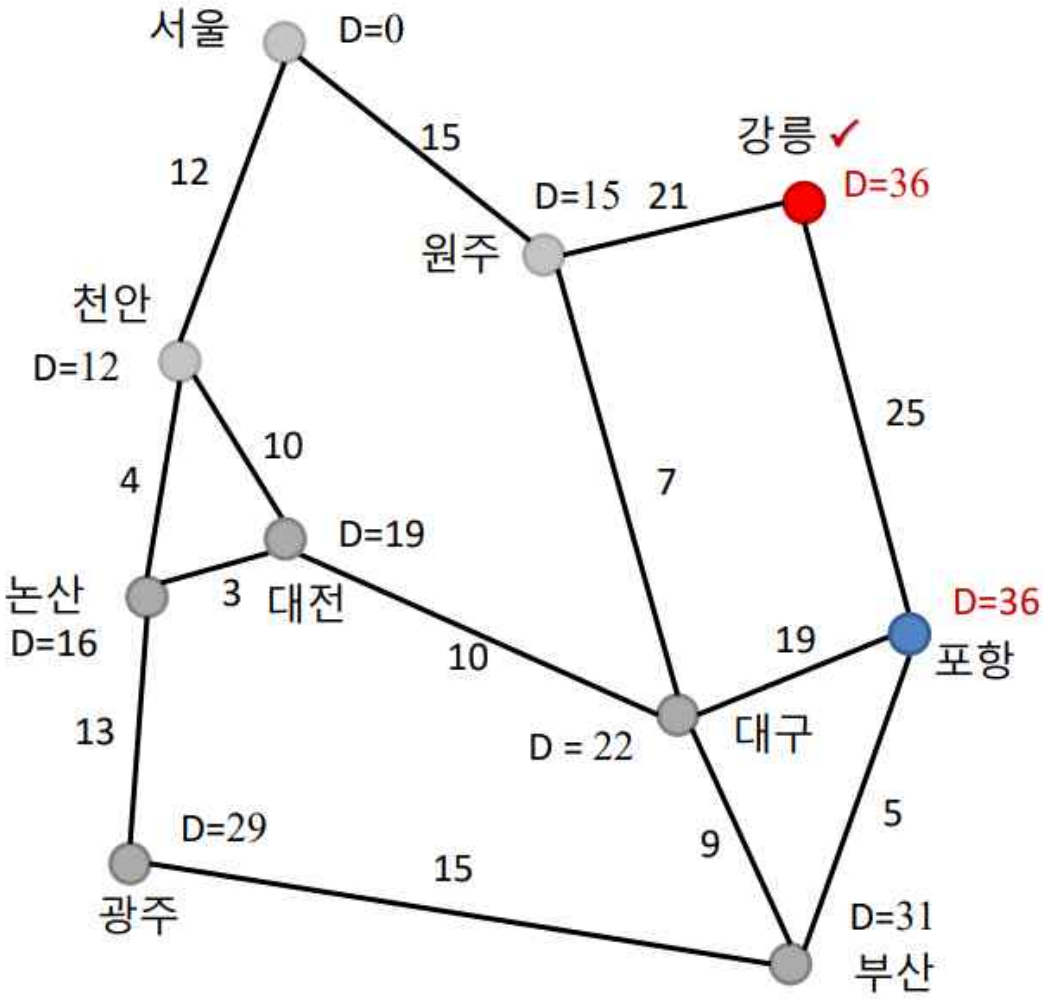

서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=∞
19
포항
10
13
D = 22
대구
5
D=29
9
15
D=∞
광주
부산

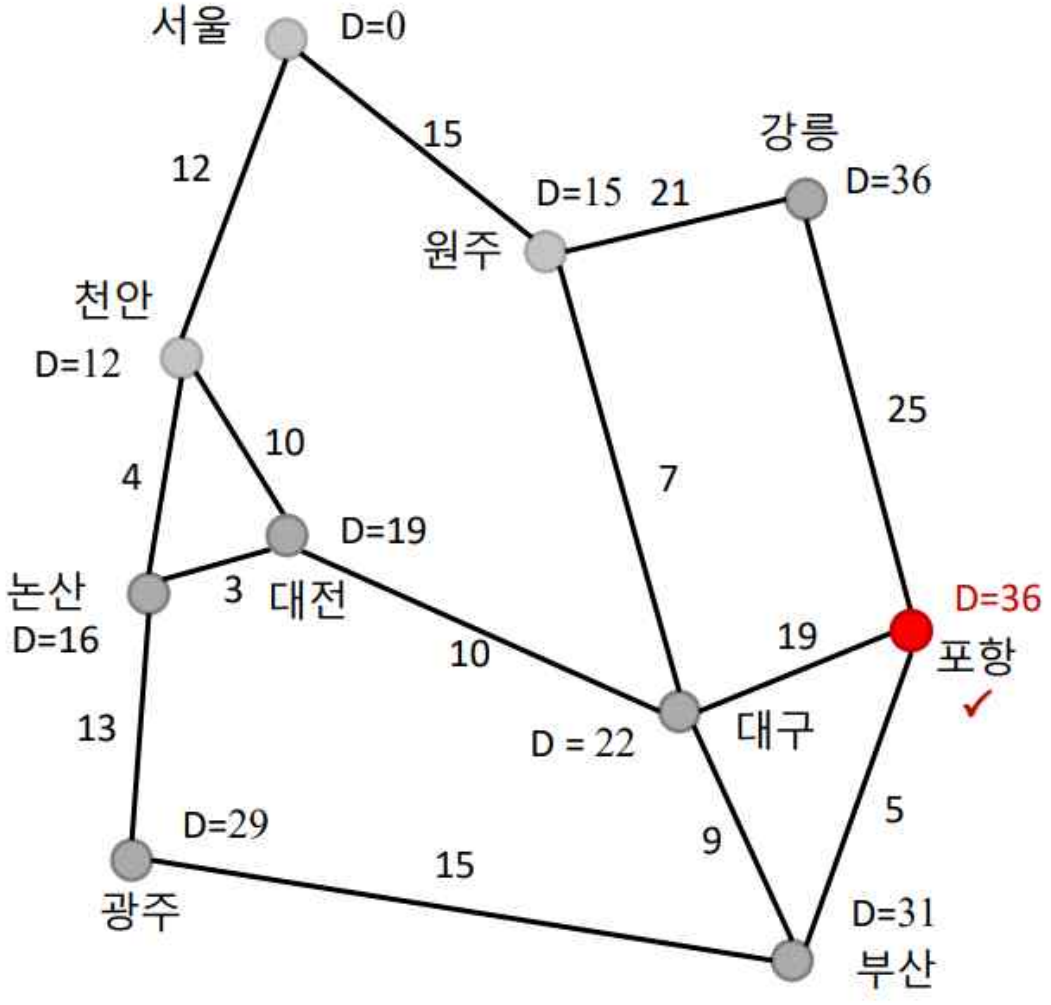

서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
D=22+19
=41
논산
D=16
3
대전
19
포항
10
13
대구
D = 22
D=29
9
5
15
D=22+9
=31
광주
부산

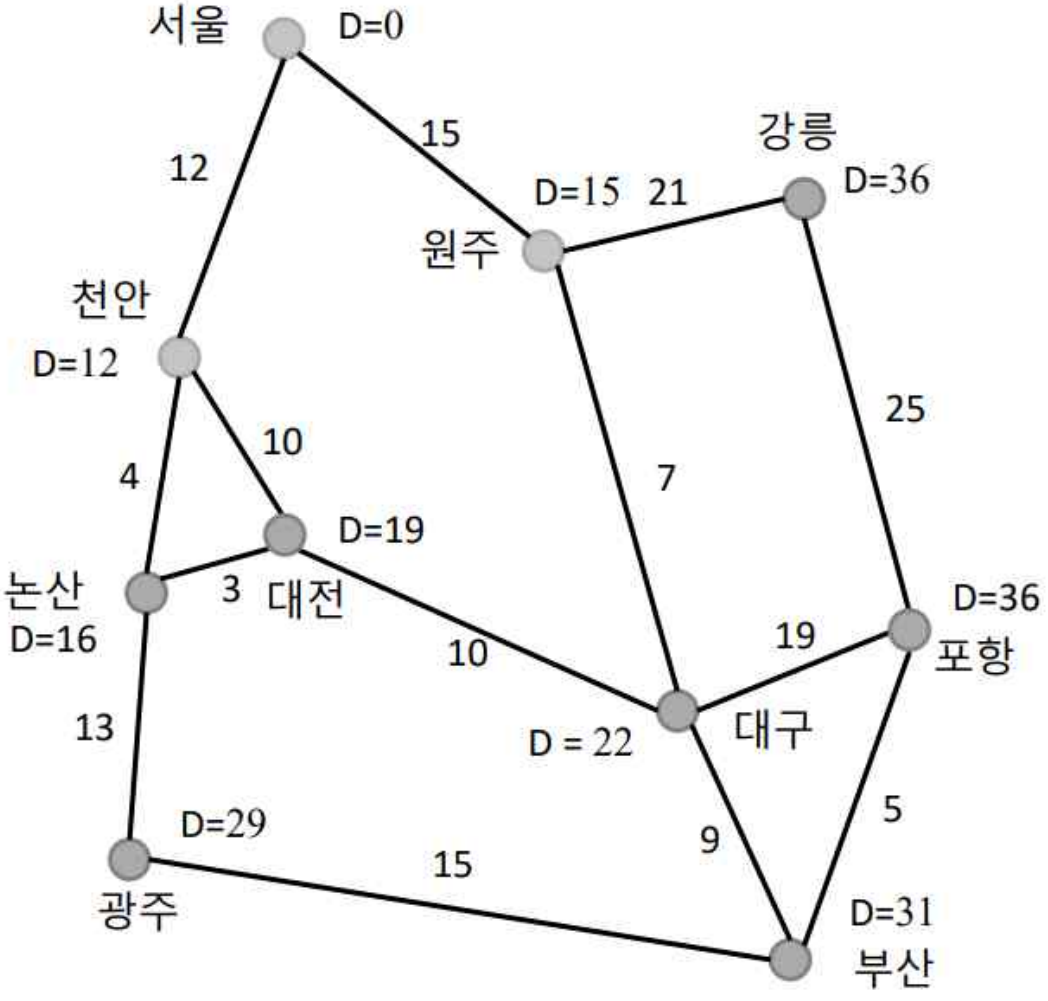

서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=41
19
10
포항
13
D = 22
대구
5
D=29
9
15
D=31
광주 ✓
부산


서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=41
19
10
포항
13
D = 22
대구
5
D=29
9
15
D=31
광주
부산 ✓


서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
4
7
D=19
D=41 ⇨
D=31+5=36
논산
D=16
3
대전
19
포항
10
13
대구
D = 22
5
9
D=29
15
광주
D=31
부산


서울
D=0
15
12
강릉 ✓
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=36
19
포항
10
13
D = 22
대구
5
D=29
9
15
D=31
광주
부산


서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
4
7
D=19
논산
3
대전
D=36
D=16
19
10
포항
13
D = 22
대구
5
D=29
9
15
D=31
광주
부산


서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
4
7
D=19
논산
3
대전
D=36
D=16
19
10
포항
13
D = 22
대구
5
D=29
9
15
D=31
광주
부산

Thank You!

한빛아카데미 Hanbit Academy, Inc.

Copyright© 2021 Hanbit Academy, Inc.
All rights reserved.